

Explainable Cervical Cell Classification: A Comparison of Preprocessing-Enhanced Machine Learning and Deep Learning Approach with Grad-CAM using SIPaKMeD

Ayşe Ece Akkurt
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
ayseece.akkurt@agu.edu.tr

Selin Nisa Akgöl
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
selinnisa.akgol@agu.edu.tr

Muhammet Talha Akşit
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
muhammettalha.aksit@agu.edu.tr

Dataset Used: SIPaKMeD (Pap Smear Dataset)
Link: <https://www.cs.uoi.gr/~marina/sipakmed.html>
Description: 4049 Pap smear cell images across five classes.

dataset webpage [1] and uploaded to Google Drive for use in Google Colab. The dataset contains 4049 Pap smear cell images from five different cervical cell classes. These classes are Dyskeratotic, Koilocytotic, Metaplastic, Parabasal, and Superficial-Intermediate.

The original dataset files were provided in compressed .7z format. Each compressed file contained images from one cell class. After downloading the files, they were extracted into a new folder named SIPaKMeD_extracted. This folder kept the original five-class structure of the dataset. This organization was important because the images had to be loaded according to their class labels during preprocessing and model training.

The original cell images were in .bmp format. The dataset also included manually prepared .dat files. These .dat files contain pixel coordinate information for the nucleus and cytoplasm borders. These borders were prepared by specialists. Therefore, the dataset was useful for both segmentation-free experiments and segmentation-based experiments. In the segmentation-free part, the images were used without separating the nucleus and cytoplasm. In the segmentation-based part, the .dat files were used as reference information for nucleus and cytoplasm segmentation.

After extraction, a representative image from each class was visually inspected to verify successful data loading prior to preprocessing and model training.

After checking the dataset, all images were resized to 224×224 pixels. This size was selected because CNN-based models such as ResNet50, DenseNet121, and MobileNetV2 commonly use this input size. Therefore, resizing made the dataset more stable and easier to process.

After resizing, the images were converted from .bmp format to .png format. PNG format was preferred because it keeps image quality while using less storage space. This made the dataset easier to store and use in Google Colab.

The prepared images were then divided into training,

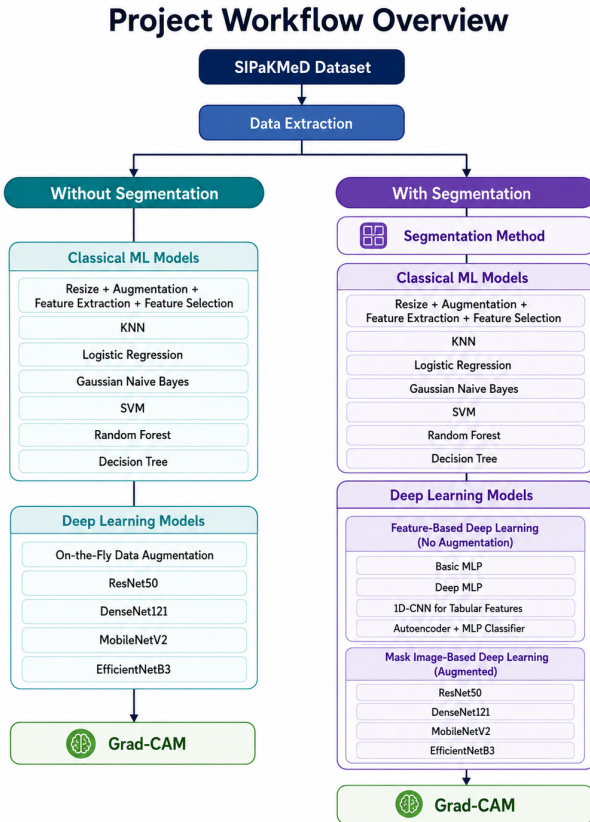


Fig. 1: Project Outline

I. DATA EXTRACTION

The dataset used in this project is the SIPaKMeD Pap Smear Cell Dataset. It was downloaded from the official

Example Image from .7z: 073.bmp

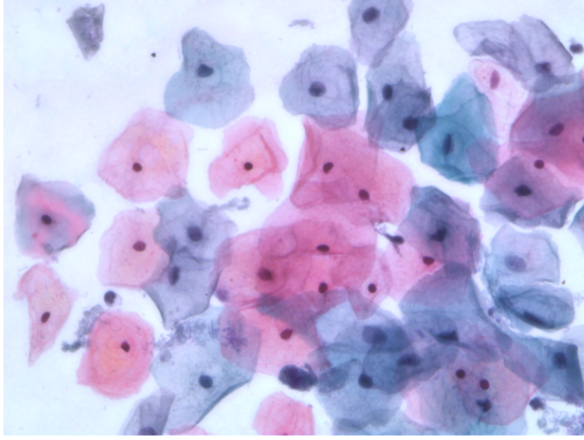


Fig. 2: Example cluster image from the SIPaKMeD dataset.

validation, and test sets. A 70/15/15 split ratio was used. The split dataset was saved in a new folder named `SIPaKMeD_split_224`. The training set was used to train the models. The validation set was used to check model performance during training and to support model selection. The test set was used only for final evaluation. The five class folders were preserved in each split, so every class was represented in the training, validation, and test sets.

TABLE I: Dataset split after data extraction and resizing.

Dataset Part	Number of Images	Purpose
Training set	2834	Model training
Validation set	607	Model validation and selection
Test set	608	Final model evaluation
Total	4049	Complete dataset

This data extraction step prepared the dataset for the next parts of the project. After this step, the same organized dataset could be used for preprocessing, feature extraction, segmentation, classical machine learning, and deep learning. In this way, all models were trained and tested on a consistent dataset structure.

II. MODEL TRAINING WITHOUT SEGMENTING NUCLEUS AND CYTOPLASM

A. Classical ML Models

1) *Resize + Augmentation + Feature Extraction & Selection*: In the first part of the non-segmented experiments, classical machine learning models were trained using handcrafted numerical features. In this approach, the raw images were not given directly to the models. This is because classical machine learning models such as KNN, SVM, Random Forest, Logistic Regression, Decision Tree, and Gaussian Naive Bayes cannot automatically learn image patterns from pixels like CNN-based deep learning models. Therefore, each image first had to be converted into a numerical feature vector.

Before feature extraction, all images were resized to 224×224 pixels. This step was important because feature extraction

should be done under the same image size conditions for every sample. If images had different sizes, extracted values such as area, edge count, texture statistics, or histogram values could become inconsistent.

Training augmentation was also applied in this part. For each original training image, one augmented version was created. The augmented images were generated temporarily in memory, and they were not saved as separate image files. After augmentation, features were extracted from both the original and augmented training images. This was done to increase the diversity of the training set and make the models more robust to small changes in image position, rotation, and scale. The validation and test sets were not augmented because they should represent clean unseen data.

As a result, the original training set contained 2834 images. After applying one augmentation for each training image, the total number of training feature rows became 5668. The validation set contained 607 rows and the test set contained 608 rows. In total, the complete feature table contained 6883 rows.

TABLE II: Feature extraction dataset summary for non-segmented images

Dataset Part	Number of Rows
Original training images	2834
Augmented training images	2834
Total training rows	5668
Validation rows	607
Test rows	608
Total rows	6883

After preprocessing and augmentation, 219 numerical features were extracted from each image. These features were selected to describe different visual properties of the cervical cell images. The extracted features included color, texture, shape, grayscale intensity, and edge-based features.

Color features were extracted from RGB and HSV color spaces. For each channel, statistical values such as mean, standard deviation, minimum, maximum, median, 25th percentile, and 75th percentile were calculated. These features were useful because staining and color distribution can be different between cervical cell classes.

Texture features were extracted using GLCM and LBP. GLCM features describe the relationship between neighboring pixels and include measurements such as contrast, dissimilarity, homogeneity, energy, correlation, ASM, and entropy. LBP features describe local texture patterns by comparing each pixel with its neighboring pixels. These texture features were included because different cell types may have different internal texture and surface appearance.

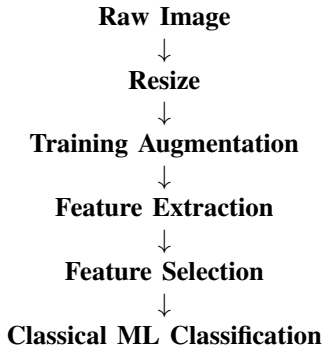
Shape and geometry features were also extracted. These features included area, perimeter, circularity, aspect ratio, extent, solidity, equivalent diameter, eccentricity, and Hu moments. These values describe the general morphology of the detected cell region. In addition, grayscale intensity features and edge-based features such as Canny edge count, edge density, Sobel

statistics, and Laplacian variance were extracted. Edge features were useful because cell boundaries and internal structures may differ between classes.

After feature extraction, feature selection was applied. Although 219 features were extracted, not all of them were expected to be equally useful. Some features may provide strong class information, while some may be repetitive or noisy. For this reason, SelectKBest with Mutual Information was used. Mutual Information measures how strongly each feature is related to the class label. In simple terms, a feature gets a higher score if it helps distinguish the five cell classes better.

Using the Mutual Information scores, four feature sets were prepared: Top 50, Top 100, Top 150, and All 219 features. This allowed us to test whether using fewer selected features could improve or simplify the classification process. However, the results showed that the best classical machine learning model used all 219 features.

The classical machine learning pipeline can be summarized as follows:



2) *KNN*: K-Nearest Neighbors was used as one of the classical baseline models. KNN classifies a test sample by looking at the closest training samples in the feature space. In this project, each image was represented by numerical features, so KNN compared test images based on their extracted feature values.

TABLE III: KNN Model Performance with Different Feature Sets

	Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	KNN	Top 150 Features	0.935855	0.935844	0.936394	0.935924
1	KNN	All 219 Features	0.932566	0.933040	0.933154	0.932822
2	KNN	Top 100 Features	0.927632	0.927667	0.928273	0.927791
3	KNN	Top 50 Features	0.906250	0.908342	0.906927	0.907077

KNN performed well and achieved its best result with the Top 150 feature set. This means that removing some weaker features helped KNN slightly, because KNN is sensitive to irrelevant or noisy features. Since KNN depends on distance calculations, unnecessary features can affect the distance between samples. Therefore, feature selection was useful for this model.

The best KNN result was 93.59% test accuracy and 0.9359 Macro F1-score. This showed that KNN could classify the cells successfully, but it did not outperform SVM or Random Forest.

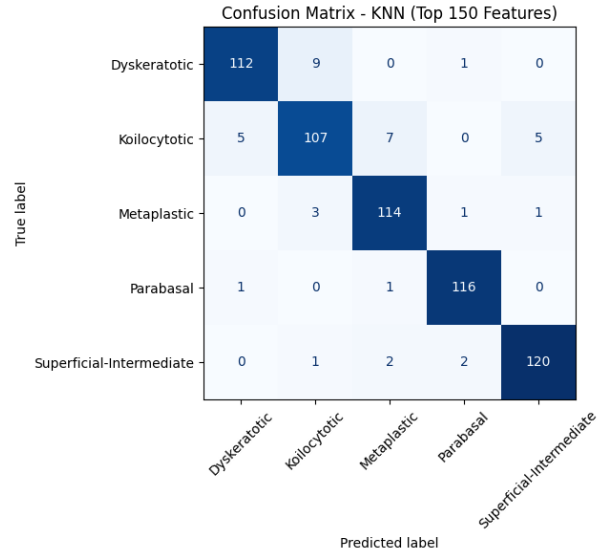


Fig. 3: KNN confusion matrix for 150 features

Figure 3 shows the confusion matrix of the KNN model with the Top 150 selected features. This figure was included because KNN obtained its best performance with this feature subset. The confusion matrix helps us see not only the final accuracy, but also which classes were predicted correctly and which classes were confused with each other. In this matrix, most values are located on the main diagonal, which means that the majority of test images were classified correctly. However, there are still some mistakes between Koilocytotic and Metaplastic samples. This is meaningful because these classes can have similar visual properties in terms of color, texture, and cell appearance. Therefore, this result shows that feature selection improved KNN, but distance-based classification still has difficulty when two classes are close to each other in the feature space.

3) *Logistic Regression*: Logistic Regression was used as a simple linear classification model. It tries to separate classes using linear decision boundaries. This model was included to observe how well a simpler linear model could perform on the extracted feature set.

TABLE IV: Logistic Regression Model Performance with Different Feature Sets

	Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	Logistic Regression	Top 150 Features	0.925987	0.926507	0.926290	0.926325
1	Logistic Regression	All 219 Features	0.924342	0.924762	0.924520	0.924512
2	Logistic Regression	Top 100 Features	0.919408	0.919611	0.919436	0.919332
3	Logistic Regression	Top 50 Features	0.884868	0.887525	0.885135	0.885798

The best Logistic Regression result was obtained using the Top 150 features. This result suggests that feature selection helped remove less useful features and made the linear classifier more stable. Logistic Regression achieved 92.60% test accuracy and 0.9263 Macro F1-score. Although this was lower than SVM, Random Forest, and KNN, it still showed that the extracted features carried meaningful class information.

4) *Gaussian Naive Bayes*: Gaussian Naive Bayes was tested as a simple probabilistic model. This model assumes

that features are conditionally independent given the class label. In real image feature data, this assumption is usually not fully true because color, texture, shape, and edge features may be related to each other.

TABLE V: Gaussian Naive Bayes Model Performance with Different Feature Sets

Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	Gaussian Naive Bayes All 219 Features	0.763158	0.767604	0.764048	0.765351
1	Gaussian Naive Bayes Top 150 Features	0.741776	0.746466	0.742648	0.743648
2	Gaussian Naive Bayes Top 100 Features	0.733553	0.734867	0.733682	0.729930
3	Gaussian Naive Bayes Top 50 Features	0.697368	0.699461	0.698101	0.694548

Gaussian Naive Bayes achieved the lowest performance among the tested classical machine learning models. It reached 76.32% test accuracy and 0.7654 Macro F1-score using all 219 features. This lower result is understandable because the extracted image features are not fully independent. Therefore, Gaussian Naive Bayes was not the most suitable model for this feature set.

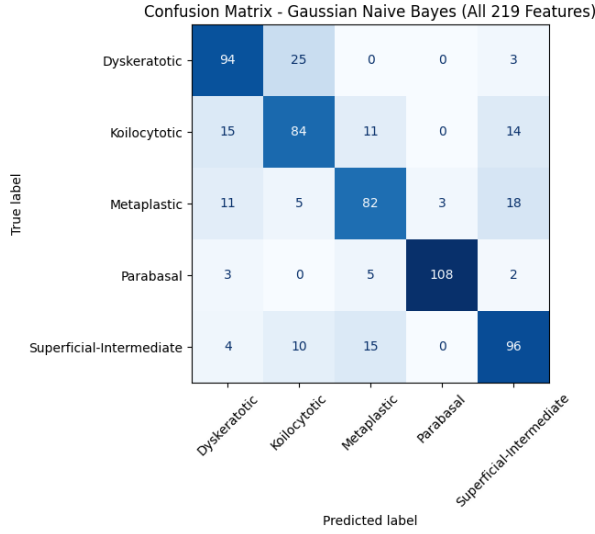


Fig. 4: Gaussian Naive Bayes confusion matrix for all features

Figure 4 shows the confusion matrix of Gaussian Naive Bayes using all 219 features. This figure was included on purpose because it represents the weakest classical machine learning model in this experiment. Unlike SVM, KNN, and Random Forest, this matrix contains many more off-diagonal values. This means that Gaussian Naive Bayes confused the cell classes more often. The reason is related to the model assumption. Gaussian Naive Bayes assumes that features are independent from each other, but our extracted image features are not fully independent. For example, color, texture, edge, and shape information can be related in cervical cell images. Because of this, Gaussian Naive Bayes could not use the feature set as effectively as the other models. This confusion matrix is useful because it clearly shows why this model had the lowest performance.

5) *SVM*: Support Vector Machine gave the best result among all classical machine learning models in the non-segmented part. SVM tries to find a strong decision boundary

between classes. This is useful when the feature space contains informative patterns but the class separation is not very simple.

TABLE VI: SVM Model Performance with Different Feature Sets

Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	SVM All 219 Features	0.949013	0.949441	0.949174	0.949179
1	SVM Top 100 Features	0.942434	0.942373	0.942696	0.942403
2	SVM Top 150 Features	0.940789	0.941181	0.941193	0.940978
3	SVM Top 50 Features	0.927632	0.928927	0.927949	0.928240

The best SVM result was obtained using all 219 features. It achieved 94.90% test accuracy, 0.9492 Macro F1-score, and 0.9491 Weighted F1-score. This showed that the full feature set was useful for SVM. Although feature selection was tested with Top 50, Top 100, and Top 150 features, these selected subsets did not improve the SVM result. Therefore, all 219 features were kept for the best SVM model.

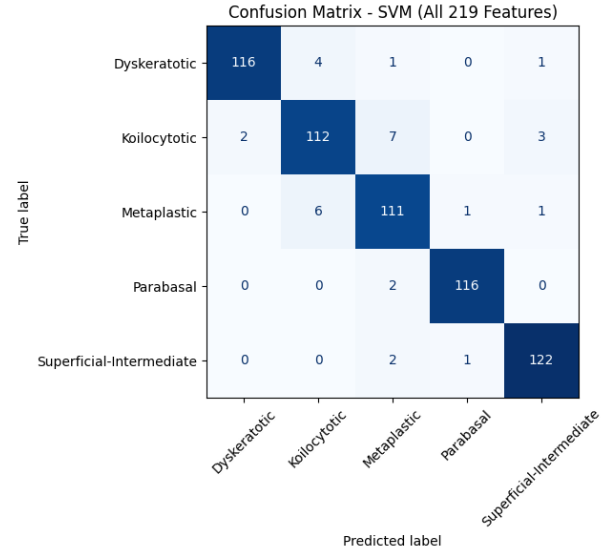


Fig. 5: SVM confusion matrix for all features

Figure 5 presents the confusion matrix of the SVM model using all 219 extracted features. This figure is one of the most important results in the classical machine learning part because SVM achieved the highest performance among all classical models. The diagonal values are very strong for all five classes, which means that the model did not perform well only for one class, but gave balanced results across the whole test set. The small number of off-diagonal values shows that the wrong predictions were limited. This supports the idea that the full handcrafted feature set was useful for SVM. In other words, combining color, texture, shape, intensity, and edge features helped the model create a better separation between the five cervical cell classes.

6) *Random Forest*: Random Forest was the second-best classical machine learning model in the non-segmented part. Random Forest combines many decision trees and makes a final prediction based on their votes. This makes it more stable than a single Decision Tree.

TABLE VII: Random Forest Model Performance with Different Feature Sets

	Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	Random Forest	All 219 Features	0.939145	0.939167	0.939575	0.939334
1	Random Forest	Top 100 Features	0.937500	0.937663	0.937677	0.937607
2	Random Forest	Top 150 Features	0.934211	0.934612	0.934575	0.934432
3	Random Forest	Top 50 Features	0.925987	0.927137	0.926277	0.926630

The best Random Forest result was obtained using all 219 features. It achieved 93.91% test accuracy and 0.9393 Macro F1-score. This showed that tree-based ensemble models were also suitable for the extracted image features. However, its performance was lower than SVM.

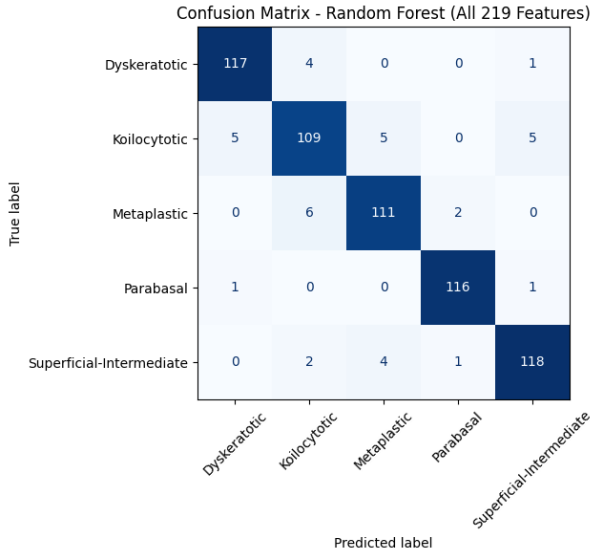


Fig. 6: Random Forest confusion matrix for all features

Figure 5 shows the confusion matrix of the Random Forest model using all 219 features. This figure was selected because Random Forest was the second strongest classical machine learning model. Similar to SVM, most samples are concentrated on the diagonal, so the model generally learned the class patterns successfully. However, the number of errors is higher compared with SVM. This tells us that Random Forest was able to use the extracted features effectively, but its decision boundaries were not as strong as SVM for this dataset. The result is still important because it shows that an ensemble of decision trees is much more reliable than a single Decision Tree model.

7) *Decision Tree*: Decision Tree was also tested as a simpler tree-based model. Unlike Random Forest, it uses only one tree. This makes the model easier to understand, but it can also make it more sensitive to overfitting.

TABLE VIII: Decision Tree Model Performance with Different Feature Sets

	Model	Feature Set	Accuracy	Precision Macro	Recall Macro	Macro F1-score
0	Decision Tree	All 219 Features	0.878289	0.880455	0.878689	0.879248
1	Decision Tree	Top 150 Features	0.875000	0.877372	0.875402	0.875428
2	Decision Tree	Top 100 Features	0.863487	0.864319	0.864078	0.863855
3	Decision Tree	Top 50 Features	0.856908	0.860269	0.857412	0.858201

The best Decision Tree result was obtained using all 219 features. It achieved 87.83% test accuracy and 0.8792 Macro F1-score. This was lower than Random Forest, which shows that using many trees together gives better generalization than relying on a single tree. Therefore, Decision Tree was useful as a simple baseline, but it was not strong enough compared with the ensemble and margin-based models.

8) *Comparison of Classical ML Models*: Table IX gives the best result of each classical machine learning model. In this comparison, SVM gave the highest performance, so it was selected as the best classical ML model for the non-segmented part. Random Forest and KNN also produced close results, which shows that the extracted features were generally informative for different types of classifiers.

The main observation is that feature selection did not improve every model in the same way. KNN and Logistic Regression benefited from the Top 150 feature set, while SVM and Random Forest performed better with all 219 features. This means that some models were more sensitive to extra or weaker features, but the strongest models were able to use the full feature set effectively. Overall, the classical ML results show that the handcrafted features provided a strong baseline before moving to deep learning models.

Model	Best Set	Acc.	Prec.	Recall	Macro F1
SVM	All 219	0.9490	0.9494	0.9492	0.9492
Random Forest	All 219	0.9391	0.9392	0.9396	0.9393
KNN	Top 150	0.9359	0.9358	0.9364	0.9359
Logistic Regression	Top 150	0.9260	0.9265	0.9263	0.9263
Decision Tree	All 219	0.8783	0.8805	0.8787	0.8792
Gaussian NB	All 219	0.7632	0.7676	0.7640	0.7654

TABLE IX: Best performance comparison of classical ML models for non-segmented images

B. Deep Learning Models

In the non-segmented deep learning part, the aim was to test whether CNN-based models could learn cervical cell patterns directly from the original images. Different from the classical ML part, we did not manually extract color, texture, shape, or edge features here. The reason is that CNN models already learn these visual patterns through their convolutional layers. Therefore, this part was used to compare manual feature-based learning with automatic image-based feature learning.

Four pretrained CNN models were used: ResNet50, DenseNet121, MobileNetV2, and EfficientNetB3. These models were not trained from zero because the SIPaKMeD dataset is not large enough for training a deep CNN from scratch. Instead, ImageNet-pretrained weights were used as a starting point. This gave the models already learned low-level visual information such as edges, textures, and color transitions. Then, the models were adapted to the five cervical cell classes by training a new classification head and fine-tuning the last layers.

1) *On-the-Fly Data Augmentation*: On-the-fly augmentation was used during CNN training. This means that the training images were randomly transformed during each epoch, but

these transformed images were not saved permanently. Small rotations, shifts, zooming, and horizontal flipping were used. These transformations are reasonable for Pap smear images because the same cell type can appear in different locations, angles, and sizes.

Augmentation was applied only to the training data. The validation and test sets were kept clean. This was important because validation and test results should measure the model performance on unseen original images, not on artificially changed images.

While checking the loss curves, validation loss sometimes looked lower than training loss. This does not always mean that the model is wrong. Rosebrock explains that this can happen when the training process is harder than the validation process, especially because of augmentation, dropout, regularization, or different training and inference behavior [2]. In our case, the model learned from augmented training images, while validation was measured on clean images. Because of this, the first loss curves were not fully comparable.

To make the graphs fairer, the model was still trained with augmented batches, but the training loss shown in the graph was recalculated using the clean training set at the end of each epoch. In this way, both training loss and validation loss were measured on clean images. This change did not make the model easier to train; it only made the plotted curves easier to interpret.

2) *ResNet50*: ResNet50 was used as the first CNN backbone. It was chosen because it is a strong deep model and residual connections help the network learn without losing information through many layers. In the first stage, the ResNet50 backbone was frozen and only the classification head was trained. This showed how useful the pretrained ResNet50 features were before changing the backbone.

TABLE X: ResNet50 training strategy

Stage	Trainable Layers	Learning Rate
Feature Extraction	Classification head only	1×10^{-4}
Fine-Tuning	conv5 block + classification head	1×10^{-5}

After the first stage, the final conv5 block was unfrozen for fine-tuning. This was done because the last block contains more task-specific visual features. Earlier layers were kept frozen because they mostly learn general features such as edges and simple textures. By opening only the last block, the model could adapt to the cervical cell images without changing the whole pretrained network.

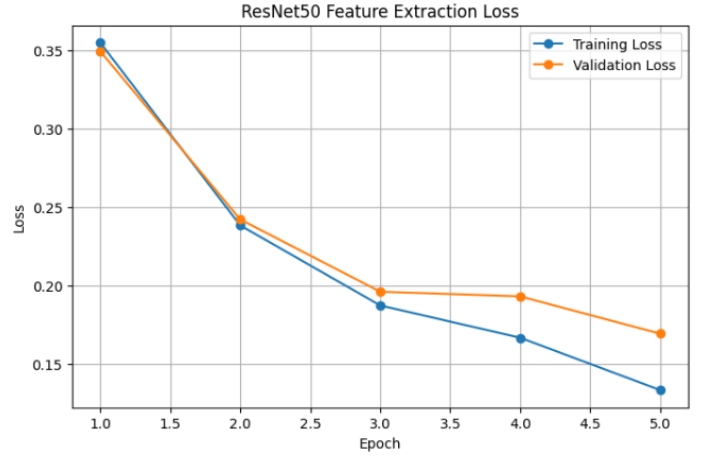


Fig. 7: ResNet50 Feature Extraction Loss Graph

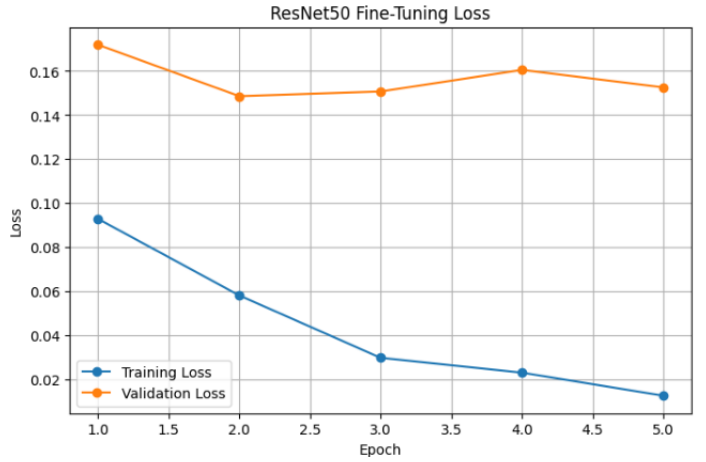


Fig. 8: ResNet50 Fine Tuning Loss Graph

The loss curves show that ResNet50 learned more effectively after fine-tuning. The frozen baseline reached 92.93% test accuracy, while the fine-tuned model reached 93.59%. This improvement shows that using only pretrained features was useful, but allowing the final block to update made the model more suitable for this dataset.

TABLE XI: ResNet50 baseline and fine-tuned performance comparison

Stage	Train	Val	Test	Acc.	Macro F1	Ep.
Baseline Frozen	0.1336	0.1695	0.2007	0.9293	0.9288	5
Fine-Tuned conv5	0.0127	0.1526	0.1808	0.9359	0.9358	10

The confusion matrix shows that ResNet50 classified most samples correctly. The strongest classes were Parabasal and Superficial-Intermediate. The most visible difficulty was around Koilocytotic and Metaplastic samples. This is important because it shows that even a strong CNN can confuse classes that have similar non-segmented visual appearance.

3) *DenseNet121*: DenseNet121 was tested because its dense connections allow later layers to reuse features learned by earlier layers. This is useful for medical images because

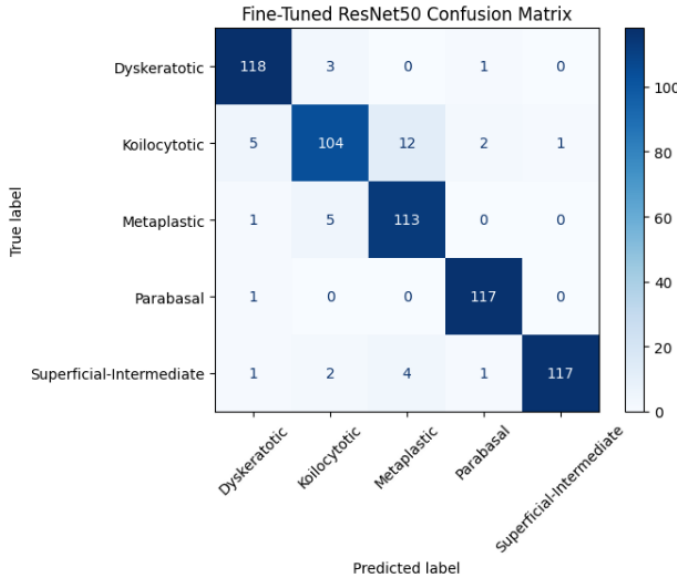


Fig. 9: ResNet50 Confusion Matrix

both small texture details and higher-level shape information can affect the classification.

TABLE XII: DenseNet121 training strategy

Stage	Trainable Layers	Learning Rate
Feature Extraction	Classification head only	1×10^{-4}
Fine-Tuning	Final dense block / conv5 related layers	1×10^{-5}

The same two-step training idea was used. First, the DenseNet121 backbone was frozen and only the classification head was trained. Then, the final dense block was opened for fine-tuning. This allowed the model to update the more class-related layers while preserving the general pretrained filters.

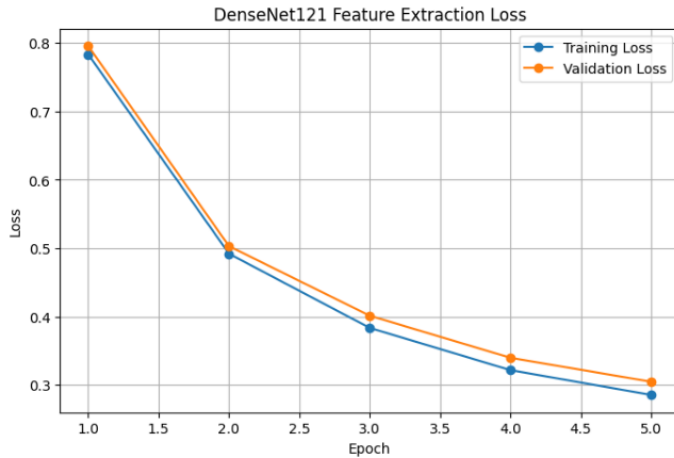


Fig. 10: DenseNet121 Feature Extraction Loss Curve

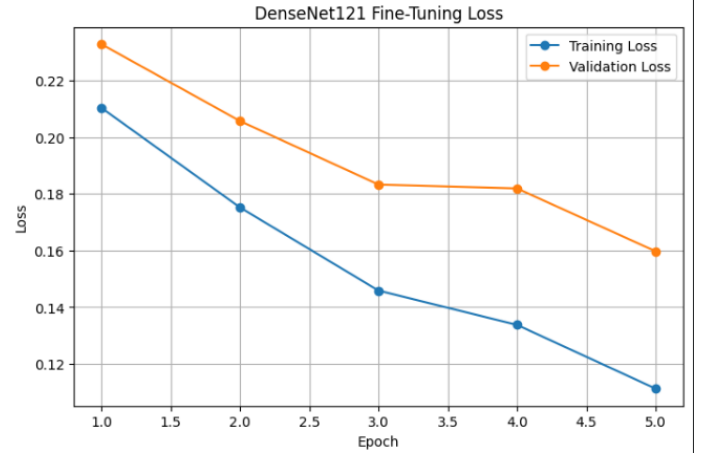


Fig. 11: DenseNet121 Fine-Tuning Loss Curve

DenseNet121 improved clearly after fine-tuning. The frozen model achieved 91.12% accuracy, while the fine-tuned version reached 93.42%. This means that the model benefited from adapting its last layers to the cervical cell dataset. Its result was very close to ResNet50, so DenseNet121 was also a strong CNN model for the non-segmented image setting.

TABLE XIII: DenseNet121 baseline and fine-tuned performance comparison

Stage	Train	Val	Test	Acc.	Macro F1	Epoch
Baseline Frozen	0.2854	0.3047	0.3093	0.9112	0.9100	5
Fine-Tuned conv5	0.1112	0.1597	0.1682	0.9342	0.9339	10

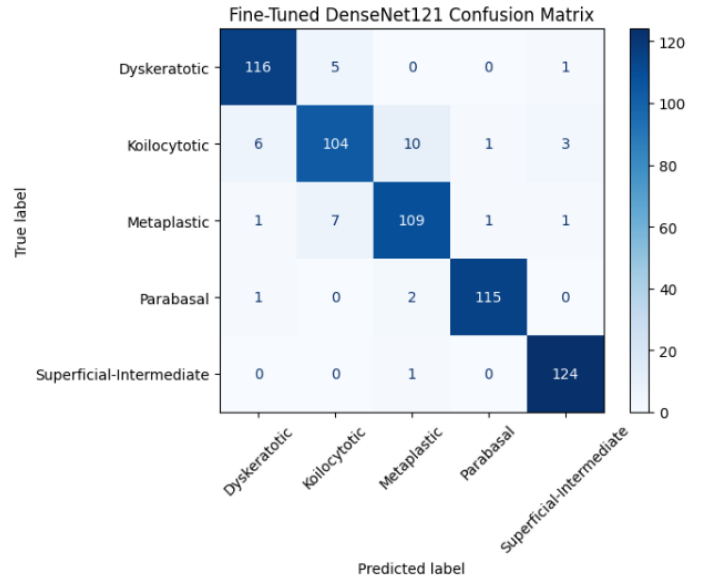


Fig. 12: DenseNet121 Confusion Matrix

The confusion matrix shows that DenseNet121 performed well for most classes. Similar to ResNet50, the main confusion was between Koilocytotic and Metaplastic cells. This repeated pattern suggests that the problem is not only model-related; these two classes are visually close in the raw image form.

4) *MobileNetV2*: MobileNetV2 was included to test a lighter CNN architecture. This model is smaller and faster than the other CNN backbones. Therefore, it was useful to see whether a compact model could still classify the cell images successfully.

TABLE XIV: MobileNetV2 training strategy

Stage	Trainable Layers	Learning Rate
Feature Extraction	Classification head only	1×10^{-4}
Fine-Tuning	block_15, block_16 + classification head	1×10^{-5}

For MobileNetV2, the final block_15 and block_16 layers were opened during fine-tuning. These blocks are near the output side of the network, so they are more related to high-level class information. Most earlier layers were kept frozen to avoid unnecessary changes in the pretrained filters.

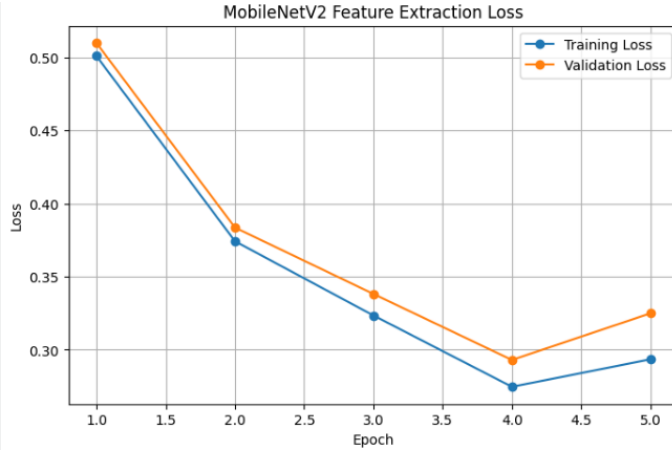


Fig. 13: MobileNetV2 Feature Extraction Loss Curve

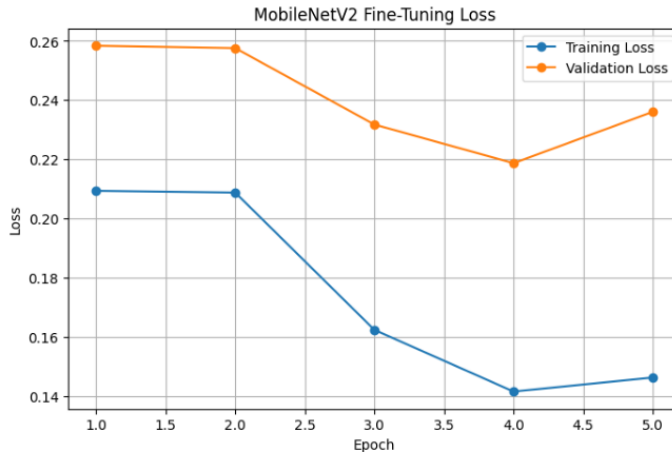


Fig. 14: MobileNetV2 Fine-Tuning Loss Curve

MobileNetV2 improved from 89.64% to 91.61% after fine-tuning. This shows that the fine-tuning step was still useful.

However, its final result was lower than the other CNN models. The main reason is probably that MobileNetV2 is designed to be efficient and lightweight, so it has less capacity to learn detailed patterns compared with larger models.

TABLE XV: MobileNetV2 baseline and fine-tuned performance comparison

Stage	Train	Val	Test	Acc.	Macro F1	Ep.
Baseline Frozen	0.2934	0.3249	0.3038	0.8964	0.8956	5
Fine-Tuned block_15/block_16	0.1463	0.2360	0.2319	0.9161	0.9156	10

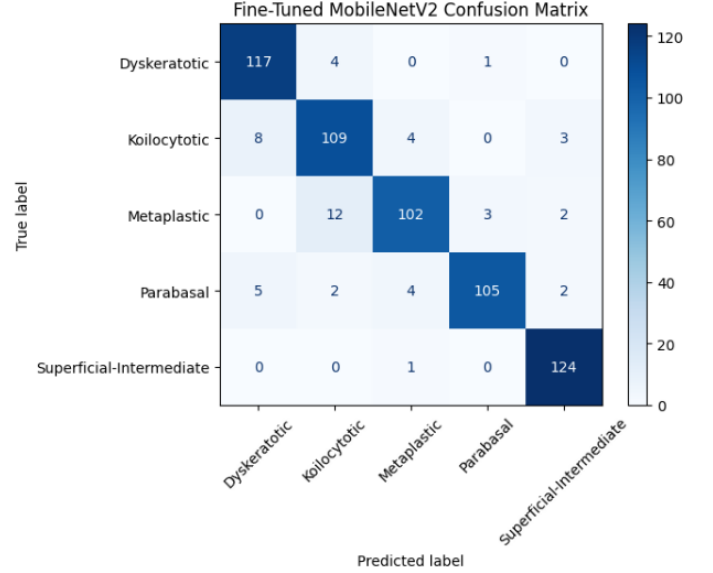


Fig. 15: MobileNetV2 Confusion Matrix

The confusion matrix supports this result. MobileNetV2 classified many samples correctly, but it produced more mistakes than ResNet50, DenseNet121, and EfficientNetB3. Therefore, MobileNetV2 was useful as an efficient baseline, but it was not the best choice when the goal was maximum accuracy.

5) *EfficientNetB3*: EfficientNetB3 was tested as the strongest CNN candidate in this part. It was selected because EfficientNet models are designed to balance input resolution, network width, and network depth. For this reason, EfficientNetB3 was trained with 300×300 input images instead of 224×224 . This allowed the model to use more image detail.

TABLE XVI: Extra EfficientNetB3 fine-tuning strategy

Stage	Trainable Layers	Learning Rate
Normal Fine-Tuning	block6, block7, top_conv	1×10^{-5}
Extra Fine-Tuning	block5, block6, block7, top_conv	5×10^{-6}

First, EfficientNetB3 was trained with the backbone frozen. Then, `block6`, `block7`, and `top_conv` were opened for fine-tuning. This normal fine-tuning improved the model, but the result was still close to ResNet50 and DenseNet121. Because of this, an extra fine-tuning step was tested.

In the extra fine-tuning step, `block5` was also opened. This gave the model more flexibility to adjust to the SIPaKMeD images. Since opening more layers can damage pretrained weights, the learning rate was reduced to 5×10^{-6} . This made the updates smaller and safer.

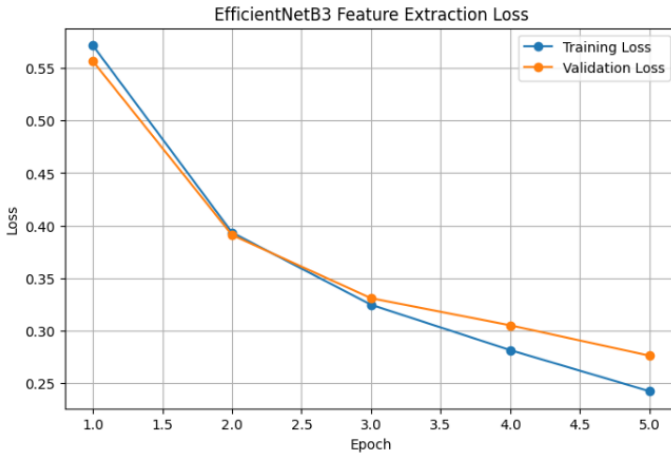


Fig. 16: EfficientNetB3 Feature Extraction Loss Curve

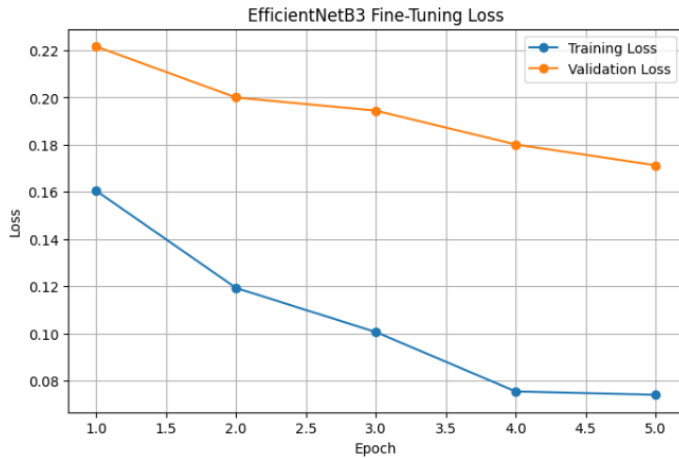


Fig. 17: EfficientNetB3 Fine-Tuning Loss Curve

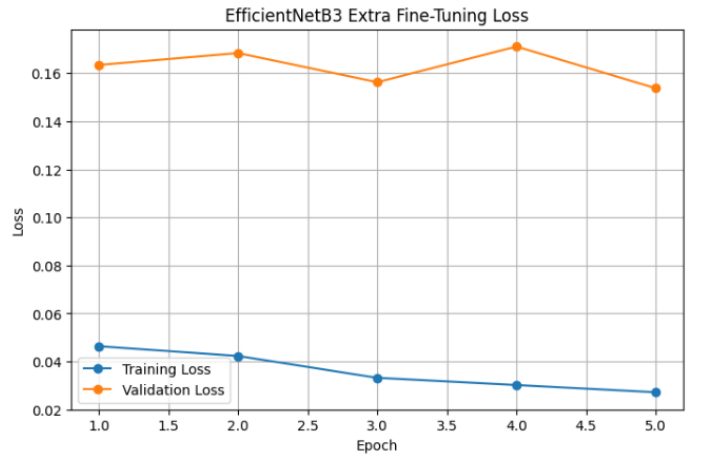


Fig. 18: EfficientNetB3 Extra Fine-Tuning Loss Curve

The extra fine-tuning step gave the best result. The normal fine-tuned EfficientNetB3 reached 93.26% accuracy, while the extra fine-tuned model reached 94.57%. This means that opening `block5` helped the model learn more useful cell-specific representations. The lower learning rate was important because it allowed the model to adapt without making unstable updates.

TABLE XVII: EfficientNetB3 performance comparison

Stage	Train	Val	Test	Acc.	Macro F1	Ep.
Baseline	0.2422	0.2761	0.3183	0.8914	0.8904	5
FT-1	0.0741	0.1713	0.1982	0.9326	0.9321	10
FT-2	0.0272	0.1538	0.1716	0.9457	0.9454	15

FT-1: `block6/block7` fine-tuning. FT-2: extra fine-tuning with `block5/block6/block7`.

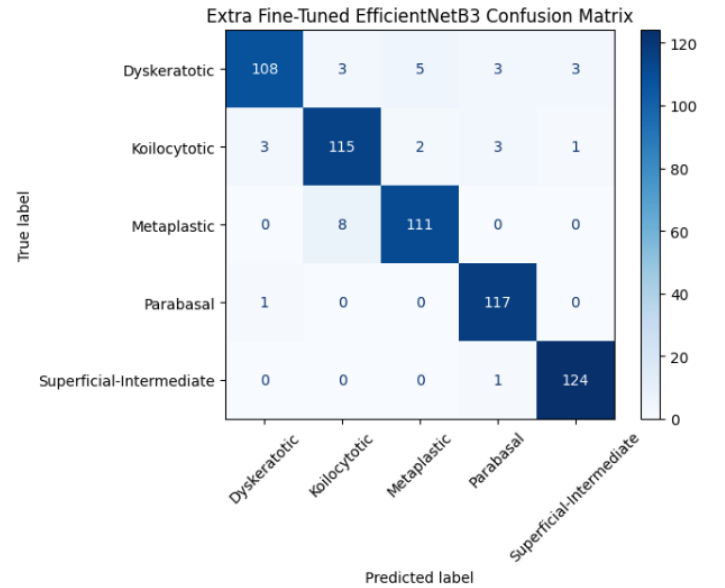


Fig. 19: EfficientNetB3 Confusion Matrix

The confusion matrix shows that EfficientNetB3 classified most test images correctly. Parabasal and Superficial-

Intermediate samples were especially strong. Some errors still remained between Dyskeratotic, Koilocytotic, and Metaplastic cells. This shows that the model learned strong image features, but non-segmented images still contain visual similarities that make some classes difficult.

6) *Comparison of Deep Learning Models:* Table XVIII shows that the best non-segmented CNN model was EfficientNetB3 after extra fine-tuning. ResNet50 and DenseNet121 followed with very close results, while MobileNetV2 gave the lowest score among the CNN models. The overall result shows that pretrained CNNs can classify non-segmented cervical cell images successfully. However, the fine-tuning choice was important. EfficientNetB3 became the best model only after opening one additional upper block with a smaller learning rate.

TABLE XVIII: Final comparison of non-segmented deep learning models

Model	Stage	Train	Val	Test	Acc.	Macro F1
EfficientNetB3	Extra FT	0.0272	0.1538	0.1716	0.9457	0.9454
ResNet50	FT	0.0127	0.1526	0.1808	0.9359	0.9358
DenseNet121	FT	0.1112	0.1597	0.1682	0.9342	0.9339
EfficientNetB3	FT	0.0741	0.1713	0.1982	0.9326	0.9321
MobileNetV2	FT	0.1463	0.2360	0.2319	0.9161	0.9156

C. Grad-CAM

Grad-CAM was used to understand where the deep learning models focused while making predictions on the non-segmented test images. Accuracy alone only shows whether the prediction is correct or wrong. However, it does not show whether the model learned meaningful cell regions or whether it was affected by background, staining artifacts, or irrelevant image parts. Therefore, Grad-CAM was added as an explainability step.

Since this experiment was performed on non-segmented images, there was no manual ground-truth cell mask for localization. For this reason, an *estimated cell region* was generated automatically and used only for attention analysis. This estimated region was not used as a real segmentation ground truth. It was only used to approximately check how much Grad-CAM attention was located inside the cell area and how much was located in the background.

For each model, Grad-CAM was applied to the full test set. Since the test set contained 608 images and five deep learning models were evaluated, the Grad-CAM analysis produced 3040 attention records in total. The example figures shown in the report are only representative examples; the numerical tables were calculated from the full test set.

The following values were used in the Grad-CAM analysis:

- **Cell Attention (%)**: average amount of Grad-CAM attention inside the estimated cell region.
- **Background Attention (%)**: average amount of Grad-CAM attention outside the estimated cell region.
- **Hot Region Inside Cell (%)**: percentage of the strongest Grad-CAM region located inside the estimated cell region.

Table XXIV shows that EfficientNetB3_Extra gave the strongest overall result. It achieved the highest test accuracy with 94.57% and also had the highest mean confidence with 96.22%. More importantly, its *Hot Region Inside Cell* value was 79.41%, which means that the strongest activation area was mostly located inside the estimated cell region. This supports that the best model was not only accurate, but also usually focused on meaningful cell areas.

The other CNN models also mostly focused on the estimated cell region. Their cell attention values were generally around 60–66%, while background attention stayed around 34–40%. These values are not expected to be perfect because the images were not manually segmented and Grad-CAM heatmaps are coarse. Still, the results show that the models did not only depend on random background regions.

To understand the mistakes better, correct and wrong predictions were analyzed separately. This is important because a wrong prediction does not always mean that the model looked at the wrong place. Sometimes the model looks at the correct cell region, but still predicts a wrong class because some cell types are visually similar.

Table XX shows that correct predictions generally had higher confidence than wrong predictions. This is expected because the models were more certain when they classified the images correctly. Also, for every model, the *Hot Region Inside Cell* value was higher in correct predictions than in wrong predictions. This means that when the model was correct, its strongest attention region was more likely to stay inside the cell.

The wrong predictions showed two main error types. The first type was **background-driven error**. In this case, the Grad-CAM activation moved away from the important cell region and focused more on background or staining-related regions. This can mislead the model because the decision is affected by image parts that are not directly related to the cell morphology.

The second type was **cell-focused class confusion**. In this case, the model still focused on the cell region, but the predicted class was wrong. This does not mean that the model guessed randomly. It means that the model looked at a relevant region, but the visual features were not enough to separate the classes correctly. This can happen because some SIPaKMeD classes have similar cytoplasm color, nucleus appearance, texture, and cell shape. For example, Koilocytotic and Metaplastic cells can sometimes look visually close in terms of staining and cell structure. Also, low contrast, overlapping visual patterns, image blur, and staining variation can make the boundary between classes harder for the model.

Representative correct and wrong Grad-CAM examples are shown in Figures 20 and 21. These figures include the original cell image, the estimated cell region, and the Grad-CAM overlay.

TABLE XIX: Model-level Grad-CAM attention summary for non-segmented test images

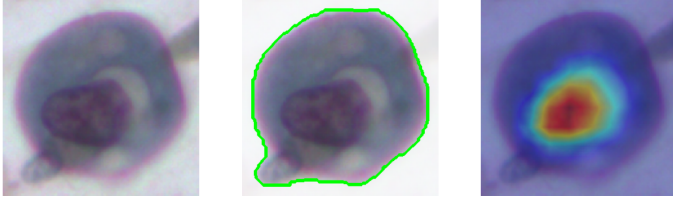
Model	Test Images	Accuracy (%)	Cell Att. (%)	Bg. Att. (%)	Hot Inside Cell (%)
EfficientNetB3_Extra	608	94.57	66.47	33.53	79.41
ResNet50	608	93.59	64.58	35.42	75.92
DenseNet121	608	93.42	59.98	40.02	71.24
EfficientNetB3	608	93.26	65.84	34.16	78.24
MobileNetV2	608	91.61	66.30	33.70	77.22

TABLE XX: Grad-CAM comparison of correct and wrong predictions on non-segmented test images

Model	Prediction Type	Num. Images	Mean Cell Att.	Mean Bg. Att.	Mean Hot Inside Cell
DenseNet121	Wrong	40	0.5692	0.4308	0.5635
DenseNet121	Correct	568	0.6019	0.3981	0.7229
EfficientNetB3	Wrong	41	0.6286	0.3714	0.6774
EfficientNetB3	Correct	567	0.6606	0.3394	0.7900
EfficientNetB3_Extra	Wrong	33	0.5864	0.4136	0.6258
EfficientNetB3_Extra	Correct	575	0.6692	0.3308	0.8038
MobileNetV2	Wrong	51	0.6432	0.3568	0.6937
MobileNetV2	Correct	557	0.6648	0.3352	0.7794
ResNet50	Wrong	39	0.6881	0.3119	0.7239
ResNet50	Correct	569	0.6429	0.3571	0.7616

Correct Grad-CAM Examples on Non-Segmented Images

Model: EfficientNetB3_Extra | True: Koilocytotic | Pred: Koilocytotic | Correct | Conf: 0.90
Original Cell Segmented / Estimated Cell Region Grad-CAM Overlay



Model: ResNet50 | True: Koilocytotic | Pred: Koilocytotic | Correct | Conf: 0.73

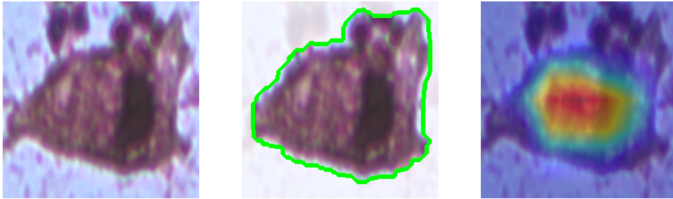
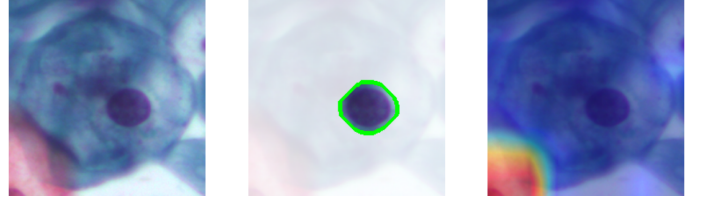


Fig. 20: Representative correct Grad-CAM examples on non-segmented images. The models predicted the correct class and the Grad-CAM activation mostly overlapped with the estimated cell region.

Wrong Grad-CAM Examples on Non-Segmented Images

Model: ResNet50 | True: Metaplastic | Pred: Koilocytotic | Wrong | Conf: 0.54
Original Cell Segmented / Estimated Cell Region Grad-CAM Overlay



Model: DenseNet121 | True: Dyskeratotic | Pred: Koilocytotic | Wrong | Conf: 0.78

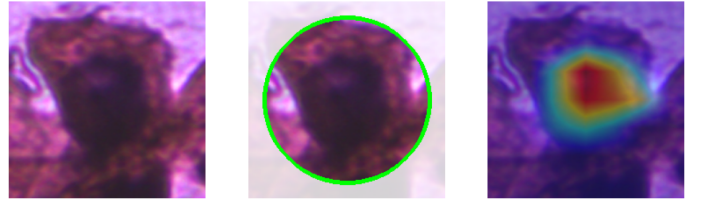


Fig. 21: Representative wrong Grad-CAM examples on non-segmented images. The examples show two error cases: background-related attention shift and cell-focused confusion between visually similar classes.

Figure 20 shows correct prediction examples. In these images, the Grad-CAM overlays are mostly located on the cell area. This supports that the models learned useful cell-related visual patterns instead of relying only on the background.

Figure 21 shows wrong prediction examples. In the first wrong example, the activation is shifted toward the background, so the prediction may have been affected by irrelevant regions. In the second wrong example, the activation is still mainly on the cell, but the class is predicted incorrectly.

This indicates that the model focused on the correct area but confused visually similar cell types.

Overall, the Grad-CAM analysis made the deep learning results more interpretable. It showed that the best model, EfficientNetB3_Extra, had both the highest accuracy and strong attention inside the estimated cell region. The analysis also showed that wrong predictions were not caused by only one reason. Some mistakes were related to background attention, while others were caused by cell-focused confusion between similar classes. Therefore, Grad-CAM helped us explain both successful predictions and failure cases in the non-segmented deep learning experiments.

III. MODEL TRAINING WITH SEGMENTING NUCLEUS AND CYTOPLASM

A. Segmentation Method

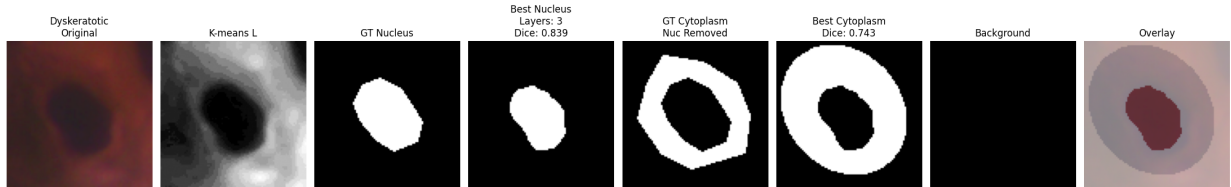
The segmentation pipeline is as follows:

- 1) Read images & apply median filter (no CLAHE because it reduced the success of this segmentation method)
- 2) Transfer from RGB to LAB and only use L channel. Due to chromatic differences, only pixel intensity will be used
- 3) Normalize L channel to bring it to the 0-255 range. Although the L channel is already represented in the range of 0–255 in OpenCV, min-max normalization was applied to stretch the intensity distribution of each image to the full dynamic range. This reduces illumination variations between samples and improves the separability of intensity clusters during K-means segmentation.
- 4) Apply 20 layer k-means clustering to increase the success of segmenting the nucleus and pick a certain num-

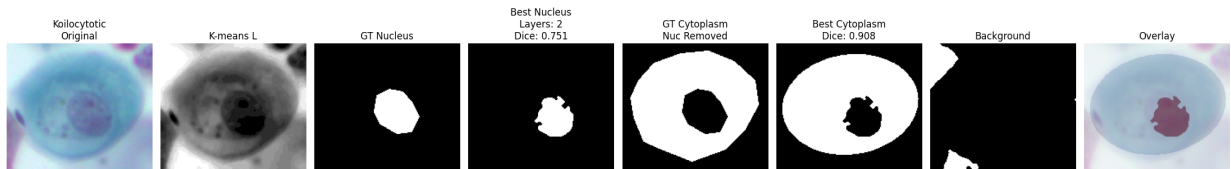
ber of darkest layers (nucleus is usually the darkest part of the image) that increase the dice score of the nucleus mask and the ground truth masks. The number of layers are chosen after a trial of ascending combinations from the darkest 5 layers.

- 5) Keep the closest piece of mask to the center (nucleus is usually close to the center) and clean up the rest by opening and closing operations.
- 6) Evaluate the k-means mask with Dice and IoU.
- 7) Construct background mask from the lightest layer.
- 8) Construct the alternative ellipse mask in case the k-means fails because it usually fails for cytoplasm. Finds the biggest contour and fits an ellipse with `cv2.fitEllipse()`. Calculate Dice.
- 9) Compare the dice score for both the k-means and the ellipse mask. Whichever scores higher Dice will be chosen as the mask.
- 10) Remove nucleus and background masks. The cluster layers left will test for combinations with highest Dice score.
- 11) Fit ellipse mask to the failed k-means mask. Evaluate Dice score.
- 12) Compare the dice score for both the k-means and the ellipse mask. Whichever scores higher Dice will be chosen as the mask.

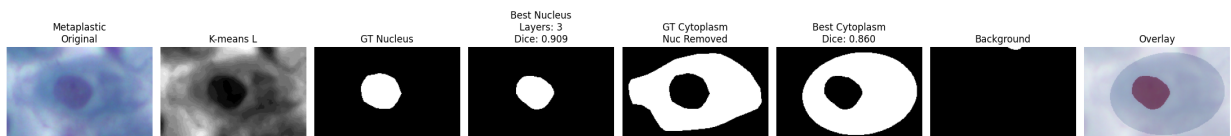
The masks are finalized after these steps and saved to a folder in .dat format for mask coordinates like the ground truth and .png for segmented images ready to be used in deep learning models. An example of segmentation is displayed in the Figure 22.



Best nucleus layers: 3 — Best cytoplasm clusters: (11,0,9,4) — Nucleus Dice: 0.8395 — Cytoplasm Dice: 0.7428



Best nucleus layers: 2 — Best cytoplasm clusters: (15,1,4,8) — Nucleus Dice: 0.7510 — Cytoplasm Dice: 0.9077



Best nucleus layers: 3 — Best cytoplasm clusters: (15,0,12,16) — Nucleus Dice: 0.9093 — Cytoplasm Dice: 0.8598

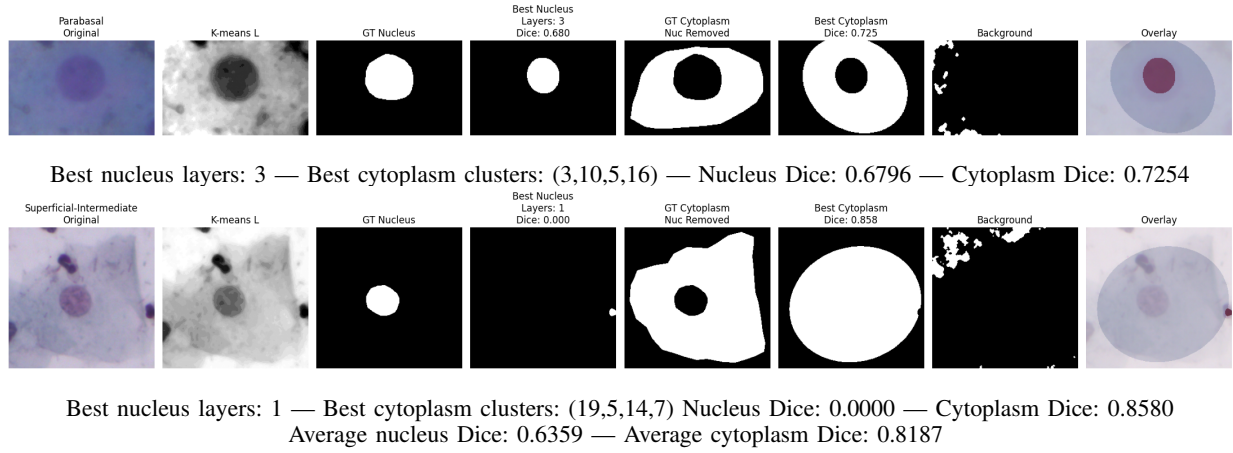


Fig. 22: Segmentation method applied on random images from all classes

After applying this method on the entire dataset:

- Average nucleus Dice: 0.8056447400701903
- Average cytoplasm Dice: 0.8456395563589089

B. Classical ML Models

1) *Resize + Augmentation + Feature Extraction & Selection*: Before training the classical machine learning models, several preprocessing and feature engineering steps were performed. First, nucleus and cytoplasm regions were segmented from each cervical cell image using the proposed segmentation pipeline. The segmentation process allowed the analysis to focus on biologically meaningful cell regions instead of the entire image, reducing the influence of the background. Since pathologists mainly evaluate cervical cells based on nuclear and cytoplasmic characteristics, separating these regions was expected to improve the quality of the extracted features.

After segmentation, handcrafted features were extracted separately from the nucleus and cytoplasm masks. Different groups of features were considered in order to describe the cells from multiple perspectives. Morphological features such as area, perimeter, circularity, solidity, eccentricity, and aspect ratio were calculated to represent the shape and geometry of the segmented regions. In addition, color-based statistics were extracted from different color spaces to capture staining and intensity variations. Texture features obtained using methods such as GLCM and LBP were also included to represent the internal texture patterns of the nucleus and cytoplasm. Furthermore, several relationship features, including nucleus-to-cytoplasm area ratios, were calculated since these measurements are commonly used in cervical cell analysis.

The complete feature extraction process produced a large number of features. However, not all extracted features contribute equally to the classification task. Therefore, a feature selection stage was applied before model training. Mutual Information was used to evaluate the relationship between each feature and the target class labels. Features with higher Mutual Information scores were considered more informative because they carried more information about the cell category.

Based on these scores, multiple feature subsets were created, including the top 50, top 100, top 150, and all available features. This allowed the effect of feature selection on classification performance to be investigated while also reducing the dimensionality of the dataset for some experiments.

2) *Model Training*: Once the feature extraction stage was completed, different machine learning models were trained using the generated feature sets. KNN, SVM, Logistic Regression, Decision Tree, Random Forest, and Gaussian Naive Bayes were tested and their results were compared. The same training and test data were used for all models to make the comparison fair. Different feature subsets obtained during feature selection were also evaluated to observe how the number of features affected the classification performance. The models were compared using accuracy, precision, recall, and F1-score values, and the best-performing model was selected based on these results.

TABLE XXI: Performance comparison of the top 5 classical machine learning models.

Model	Accuracy	Macro F1	Macro Precision	Macro Recall
Random Forest All Features	0.9737	0.9736	0.9736	0.9737
Random Forest Top 150 Features	0.9688	0.9687	0.9690	0.9687
SVM All Features	0.9655	0.9656	0.9659	0.9656
KNN All Features	0.9589	0.9588	0.9588	0.9590
Logistic Regression All Features	0.9572	0.9573	0.9575	0.9574
SVM Top 150 Features	0.9572	0.9572	0.9579	0.9573

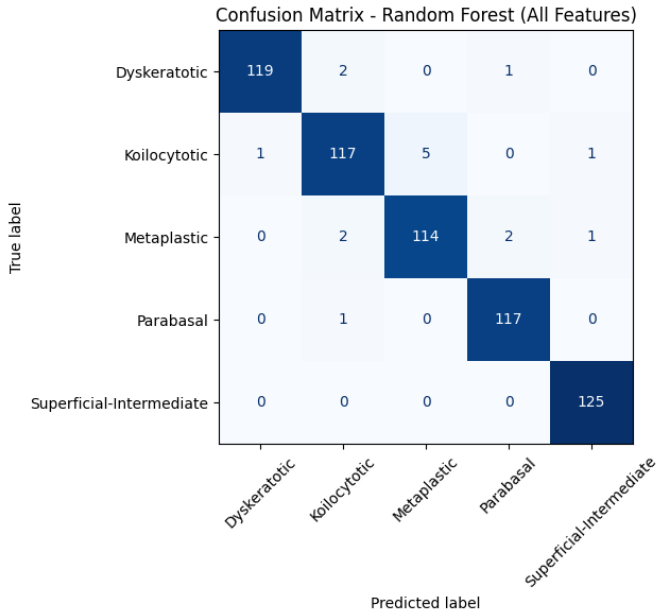


Fig. 23: Random Forest (All 716 Features) Confusion Matrix

There are a clear improvements to the same models with segmentation. Compared to all features (219) random forest without segmentation (0.9391 accuracy), the 150 features segmented version (0.9688 accuracy) performs better even while the feature number of the segmented model is higher than the non-segmented RF. These results perform this well even though the segmentation is not that successful with its Dice scores around 0.80, indicating that the improvement of segmentation can increase the accuracy of the models furthermore.

C. Deep Learning Models

1) Segmented Feature-Based Deep Learning without Augmentation: In this part, deep learning models were trained by using segmented nucleus-cytoplasm features without augmentation. The models did not use raw images directly. Instead, each cell image was represented by 716 numerical features. These features were extracted from the segmented nucleus and cytoplasm regions.

This experiment aimed to test whether deep learning models could classify cervical cell types by using only segmented feature values. This is important because the nucleus and cytoplasm are important parts of a cervical cell. Their shape, color, texture, size, and intensity values can help separate different cell classes. Therefore, segmented features were expected to give more meaningful information than using the whole image without separating cell regions.

No data augmentation was applied in this part. The models were trained only with the original segmented feature table. The complete dataset included 4049 cell images. The training set included 2834 samples, the validation set included 607 samples, and the test set included 608 samples. The same split was used for all models, so the comparison was fair.

Before training, all numerical features were standardized. This step was necessary because the feature values had different ranges. If the features are not scaled, some large-valued features can affect the model more than other features. Standardization helped all features contribute more fairly during model training.

Four different deep learning models were tested in this part: Basic MLP, Deep MLP, 1D-CNN for tabular features, and Autoencoder + MLP classifier. These models were selected to compare different learning approaches. Basic MLP was used as a simple fully connected baseline. Deep MLP was used to test whether adding more dense layers could improve performance. 1D-CNN was used to test whether convolution layers could learn local relations between neighboring feature values. Autoencoder + MLP was used to test whether the 716 features could be compressed into a smaller representation before classification.

During the first experiments, some loss curves were difficult to interpret. In some cases, validation loss looked lower than training loss. This can happen when the model behaves differently during training and validation. For example, dropout and other training-time operations are active during training but inactive during validation. Rosebrock explains that validation loss can appear lower than training loss when training and validation are measured under different conditions [2]. To make the curves more reliable, the training loss was recalculated at the end of each epoch by using `model.evaluate()` on the clean training feature set. In this way, both training loss and validation loss were measured in evaluation mode. After this correction, the curves became easier to compare.

a) Basic MLP:

The Basic MLP model was the first model tested with the segmented feature table. It used fully connected dense layers to learn the relation between the 716 input features and the five cervical cell classes. This model was selected because it is simple, fast, and suitable for numerical feature-based classification.

The Basic MLP model gave the best result among the segmented feature-based deep learning models. It achieved 96.71% test accuracy and a macro F1-score of 0.9671. This result shows that the segmented nucleus-cytoplasm features were highly useful for classification.

Fig. 24 shows the loss curve of the Basic MLP model. The training loss decreased quickly and became very low. The validation loss also decreased at the beginning, but after some epochs it stayed higher than the training loss. This means that the model learned the training data very strongly. The gap between training and validation loss may show a small overfitting tendency. However, the high test accuracy shows that the model still worked well on unseen test data.

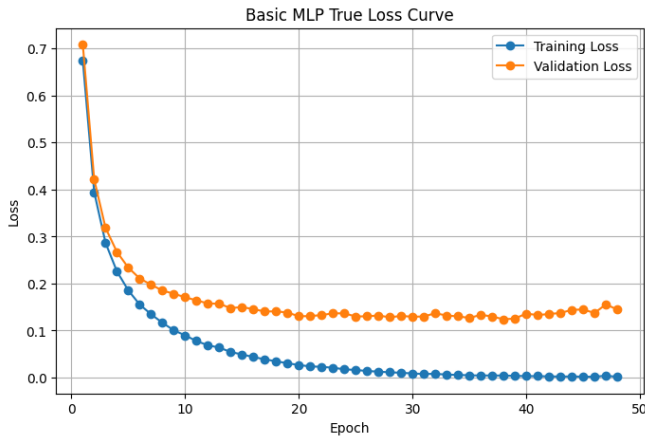


Fig. 24: True training and validation loss curves of the Basic MLP model without augmentation.

Fig. 25 shows the confusion matrix of the Basic MLP model. The model classified all Parabasal samples correctly. It also classified almost all Superficial-Intermediate samples correctly. Most errors were observed between Dyskeratotic and Koilocytotic samples, and between Koilocytotic and Metaplastic samples. This means that these classes may have similar feature patterns.

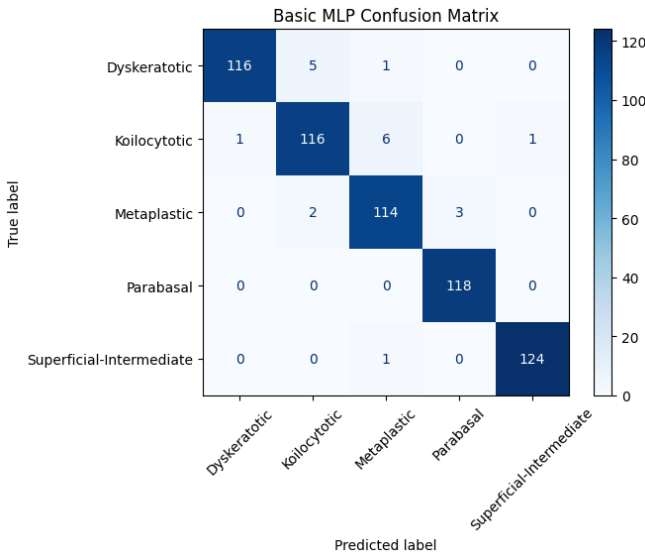


Fig. 25: Confusion matrix of the Basic MLP model using segmented nucleus-cytoplasm features without augmentation.

b) Deep MLP:

The Deep MLP model was tested to see whether a deeper fully connected network could improve the classification result. This model used more dense layers than the Basic MLP model. The purpose was to learn more complex relations between the segmented numerical features.

The Deep MLP model achieved 96.22% test accuracy and a macro F1-score of 0.9622. This result was close to the Basic MLP result, but it was slightly lower. This means that

adding more dense layers did not clearly improve the result for this feature set. Therefore, the simpler Basic MLP model was enough to learn useful information from the 716 segmented features.

Fig. 26 shows the loss curve of the Deep MLP model. The training loss decreased strongly during training. The validation loss also decreased at the beginning, but it stayed higher than the training loss after some epochs. This shows that the model learned the training data well, but there was a small generalization gap between training and validation performance.

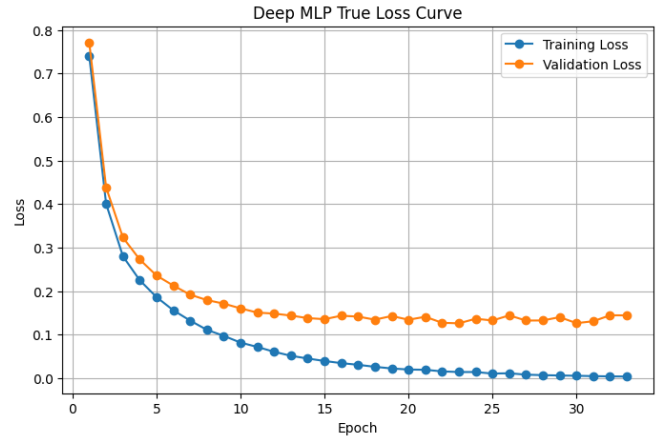


Fig. 26: True training and validation loss curves of the Deep MLP model without augmentation.

Fig. 27 shows the confusion matrix of the Deep MLP model. The model gave strong classification results for most classes. Similar to the other segmented feature-based models, some errors were observed between visually and feature-wise similar classes such as Koilocytotic and Metaplastic.

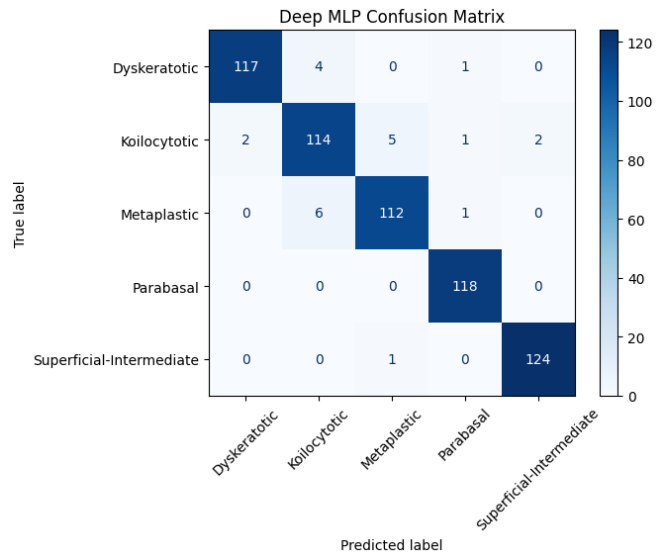


Fig. 27: Confusion matrix of the Deep MLP model using segmented nucleus-cytoplasm features without augmentation.

c) 1D-CNN for Tabular Features:

The 1D-CNN model was tested because convolutional layers can learn local relations between neighboring feature values. In this model, the 716-dimensional feature vector was used as a one-dimensional input. The aim was to check whether convolution-based learning could improve classification performance on tabular segmented features.

The 1D-CNN model achieved 96.38% test accuracy and a macro F1-score of 0.9638. This was a strong result and it was close to the Basic MLP result. Fig. 28 shows the true loss curve of the 1D-CNN model. The training and validation losses decreased strongly in the first epochs. After this point, the training loss continued to decrease, while the validation loss stayed higher and became stable. This shows a small generalization gap, but the model still achieved high test accuracy.

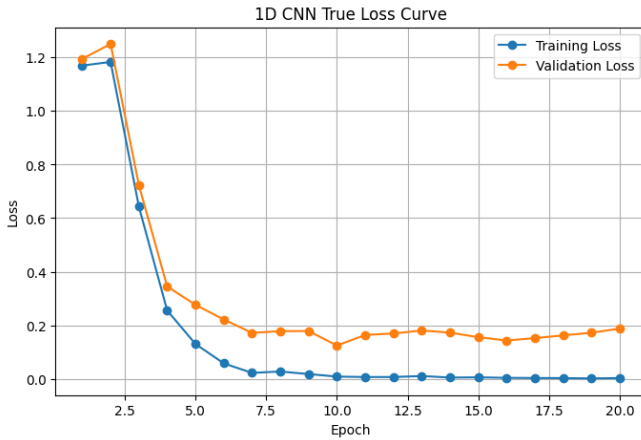


Fig. 28: True training and validation loss curves of the 1D-CNN model without augmentation.

The confusion matrix of the 1D-CNN model is shown in Fig. 29. The model performed very well for Parabasal and Superficial-Intermediate classes. Similar to the Basic MLP model, some errors were observed between Koilocytotic and Metaplastic samples. This result shows that the segmented features are useful, but some classes are still close to each other in feature space.

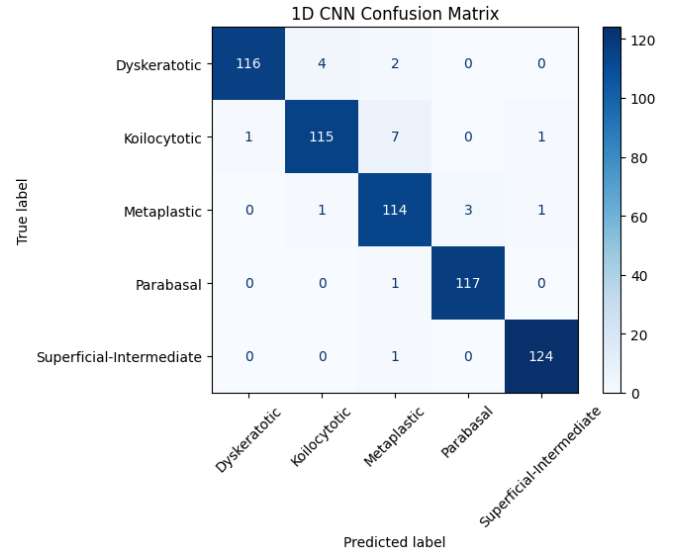


Fig. 29: Confusion matrix of the 1D-CNN model using segmented tabular features without augmentation.

d) Autoencoder + MLP Classifier:

The Autoencoder + MLP model was used to test whether the original 716 features could be compressed into a smaller representation. First, the autoencoder was trained to reconstruct the original input features. Then, the encoded features were used as input to an MLP classifier.

Fig. 30 shows the reconstruction loss of the autoencoder. Both training and validation reconstruction losses decreased during training. This means that the autoencoder learned a compact representation of the segmented feature set.

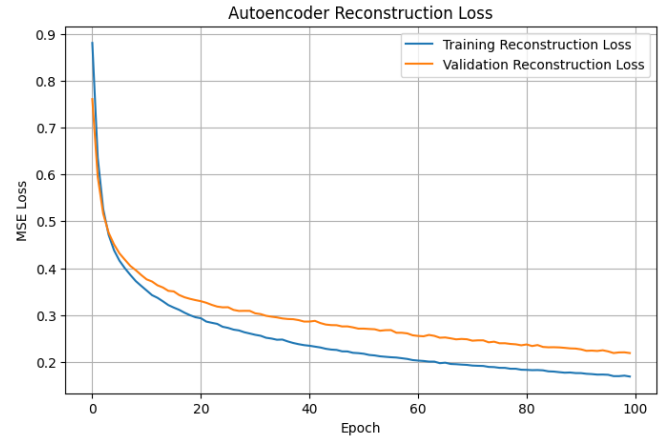


Fig. 30: Training and validation reconstruction loss curves of the autoencoder without augmentation.

After feature compression, the encoded features were used to train the MLP classifier. Fig. 31 shows the classifier loss curve. The model learned successfully, but its final test performance was slightly lower than the other deep learning models. The Autoencoder + MLP model achieved 96.05% test accuracy and a macro F1-score of 0.9606. This result shows

that the compressed features were useful, but the original 716 features contained more useful information for classification.

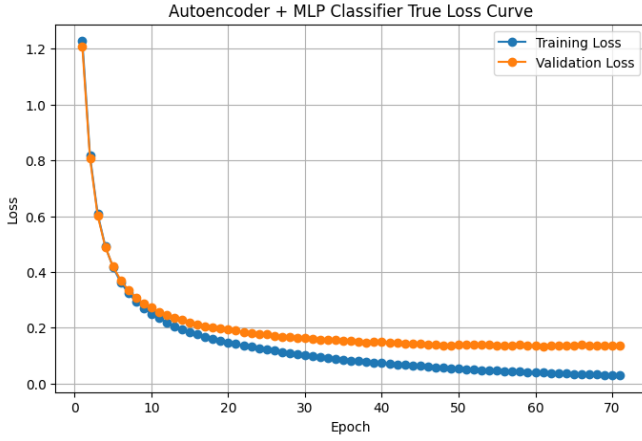


Fig. 31: True training and validation loss curves of the Autoencoder + MLP classifier without augmentation.

Table XXII shows the final comparison of the segmented feature-based deep learning models without augmentation. The Basic MLP model gave the best result. The 1D-CNN and Deep MLP models also achieved high accuracy values. The Autoencoder + MLP model achieved the lowest result among these four models, but its accuracy was still above 96%.

TABLE XXII: Performance comparison of segmented feature-based deep learning models without augmentation.

Model	Test Loss	Test Acc.	Precision	Recall	Macro F1
Basic MLP	0.2433	0.9671	0.9672	0.9673	0.9671
1D-CNN	0.4203	0.9638	0.9642	0.9640	0.9638
Deep MLP	0.1291	0.9622	0.9622	0.9623	0.9622
Autoencoder + MLP	0.1131	0.9605	0.9610	0.9605	0.9606

Overall, all four models achieved test accuracies above 96%. This result shows that segmented nucleus-cytoplasm features were very useful for cervical cell classification. Among the tested models, the Basic MLP model provided the best balance between simplicity and performance. Therefore, it was selected as the best segmented feature-based deep learning model without augmentation.

2) *Augmented Mask Image-Based Deep Learning*: The segmented masks and input images differ from both the non-segmented training and the segmented classical training. The method of segmenting stays the same however the segmented images are used as cytoplasm and nucleus. Training a model by separating them proved to be a challenge so using the segmented and cropped cell images like shown in Figure 32.

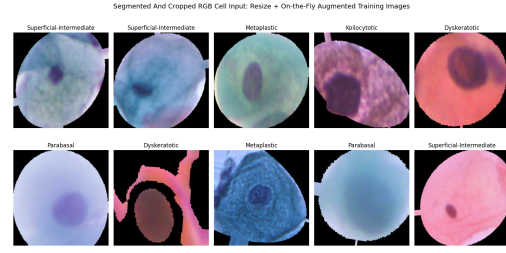


Fig. 32: Segmented, cropped and augmented images

Same method of on-the-fly augmentation applied to the non-segmented dataset is applied to the cropped and segmented cell images as well. After that, the augmented images are fed to the deep learning models below. To keep the comparison fair, the deep learning models are the same with the previously trained CNN models. The code used for training non-segmented CNN models were used only changing the directories of input and output to keep everything consistent. Therefore, the training settings are all the same and the in the following parts, only results will be presented.

a) *ResNet50*:

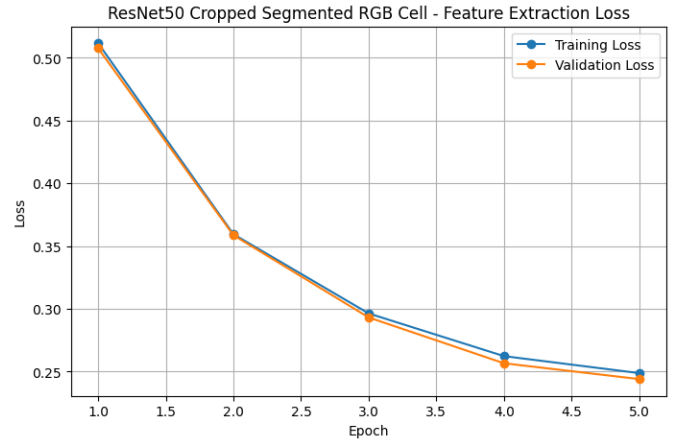


Fig. 33: ResNet50 Segmented Loss Graph

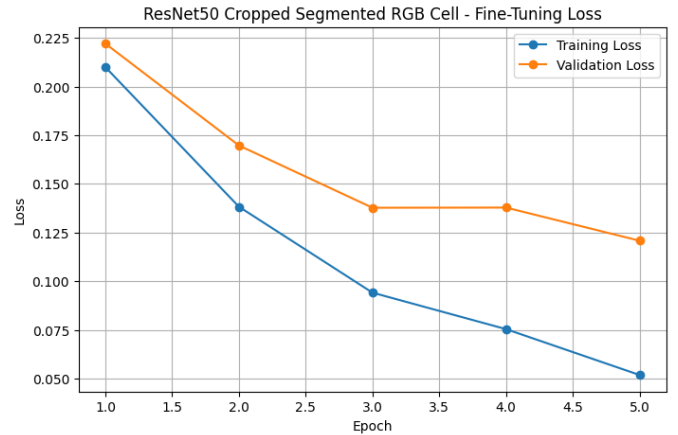


Fig. 34: Fine-Tuned ResNet50 Segmented Loss Graph

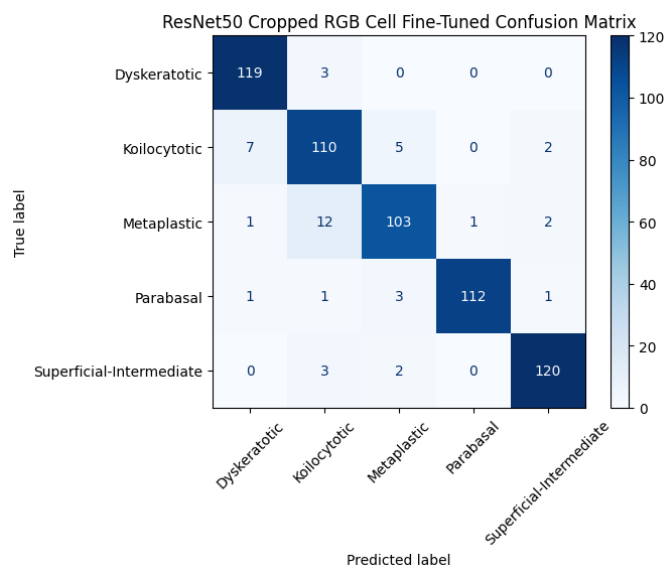


Fig. 35: ResNet50 Confusion Matrix

The training and validation loss curves dropped steadily, showing that the ResNet50 model was able to detect the key patterns in the data. Fine-tuning helped as well because losses fell for both sets, yet the gap stayed small, indicating a valid generalization without major overfitting.

A look at the confusion matrix backs this up. Our model nailed most classifications, especially for Dyskeratotic, Parabasal, and Superficial-Intermediate cells. Most errors happened when it mixed up Koilocytotic and Metaplastic classes, likely due to their similar looks. In the end, the fine-tuned ResNet50 did great at classifying those cell images.

b) DenseNet121:

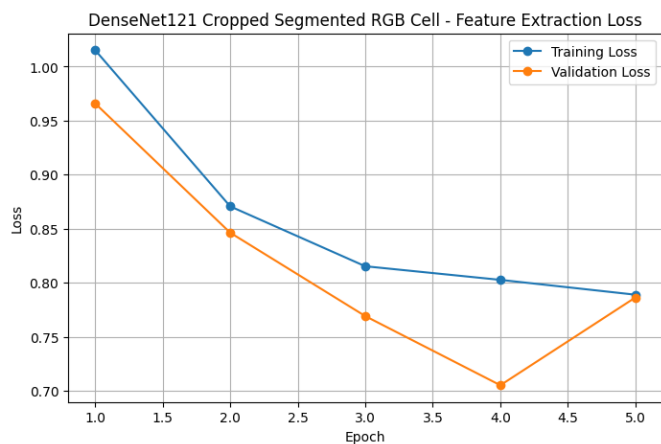


Fig. 36: DenseNet121 Segmented Loss Graph

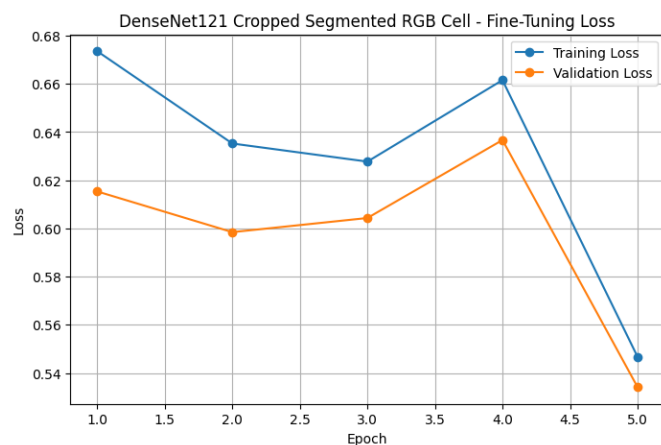


Fig. 37: Fine-Tuned DenseNet121 Segmented Loss Graph

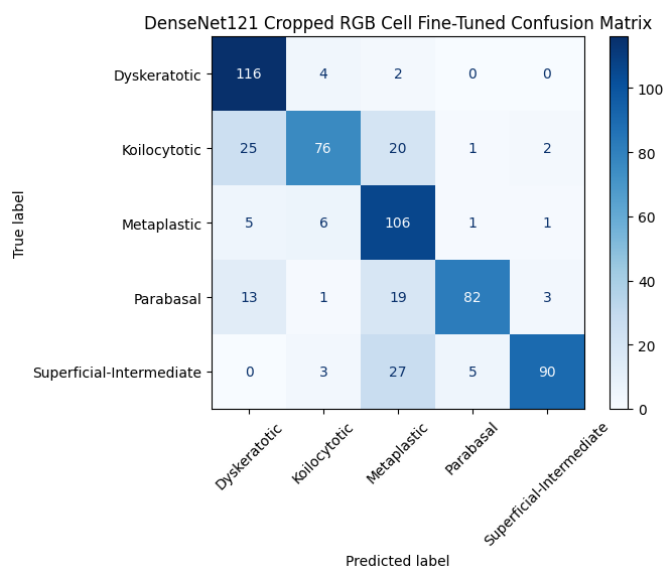


Fig. 38: DenseNet121 Confusion Matrix

The loss curves for DenseNet121 show a steady drop during feature extraction and fine-tuning, showing that the model picked up useful features from the cell images. Still, the training and validation losses were a bit jittery during fine-tuning, pointing to a less smooth training phase than some other models experienced. While the losses did eventually dip, the progress wasn't as impressive as one might hope.

Looking at the confusion matrix, DenseNet121 did pretty well with Dyskeratotic and Metaplastic classes – most samples got classified right there. Yet, the model had trouble with Parabasal, Superficial-Intermediate, and Koilocytotic cells, leading to quite a few mistakes. Specifically, Koilocytotic and Metaplastic cells got mixed up often. This suggests that even though DenseNet121 caught key cell traits, its overall performance wasn't as great as the fine-tuned ResNet50's.

c) MobileNetV2:

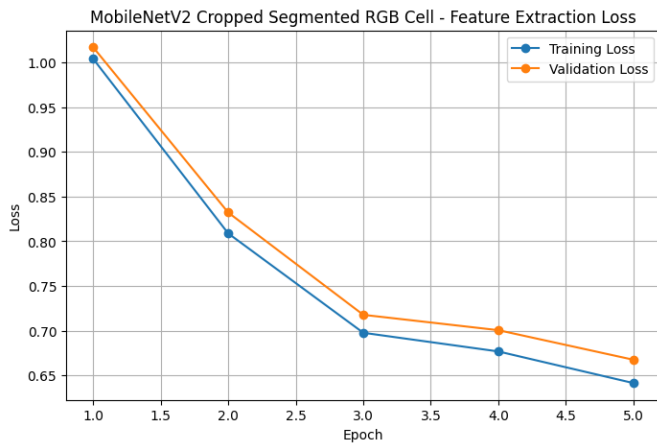


Fig. 39: MobileNetV2 Segmented Loss Graph

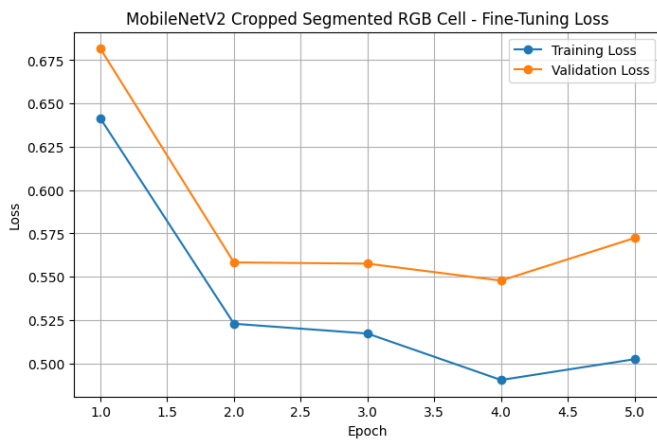


Fig. 40: Fine-Tuned MobileNetV2 Segmented Loss Graph

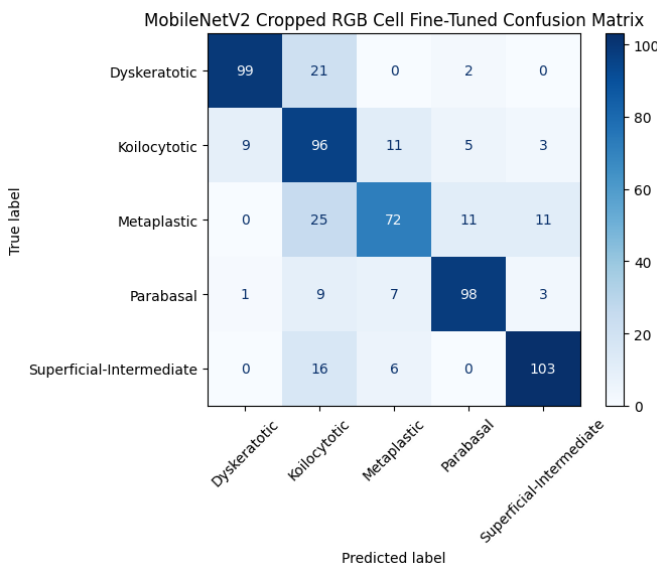


Fig. 41: MobileNetV2 Confusion Matrix

The loss curves for MobileNetV2 drop consistently throughout both stages – feature extraction and fine-tuning. This tells us the model picked up useful traits from the cell images. While fine-tuning, the training loss kept falling, yet the validation loss stayed pretty steady, showing decent generalization without serious overfitting issues.

MobileNetV2's confusion matrix highlighted solid performance in classifying Koilocytotic, Parabasal, and Superficial-Intermediate cells. Still, there were some mix-ups between Dyskeratotic and Koilocytotic cells, as well as between Metaplastic and others. Even with these mistakes, most samples got categorized correctly, proving MobileNetV2 can indeed find important features in the images and deliver strong results.

d) EfficientNetB3:

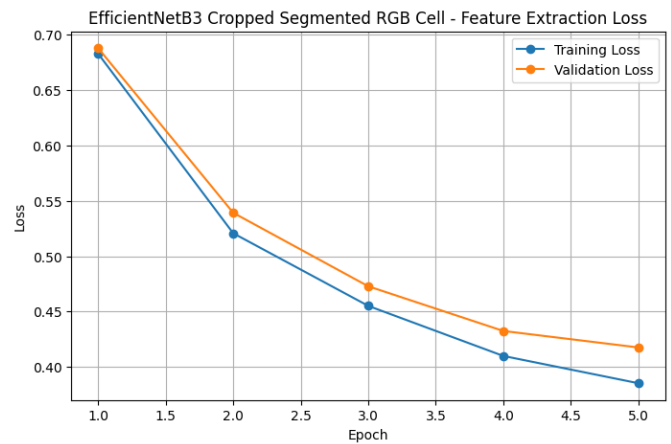


Fig. 42: EfficientNetB3 Segmented Loss Graph

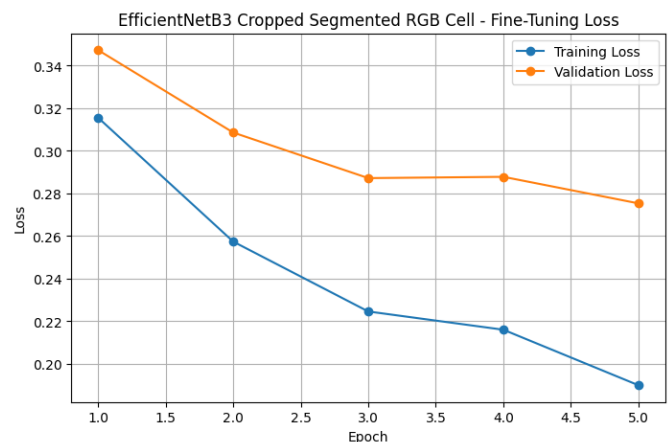


Fig. 43: Fine-Tuned EfficientNetB3 Segmented Loss Graph

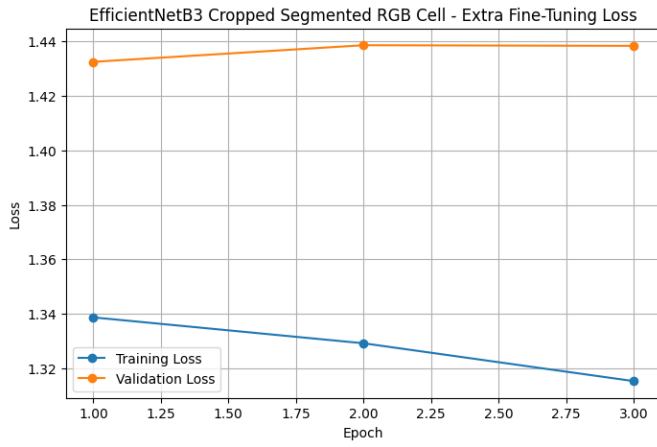


Fig. 44: Extra Fine-Tuned EfficientNetB3 Segmented Loss Graph

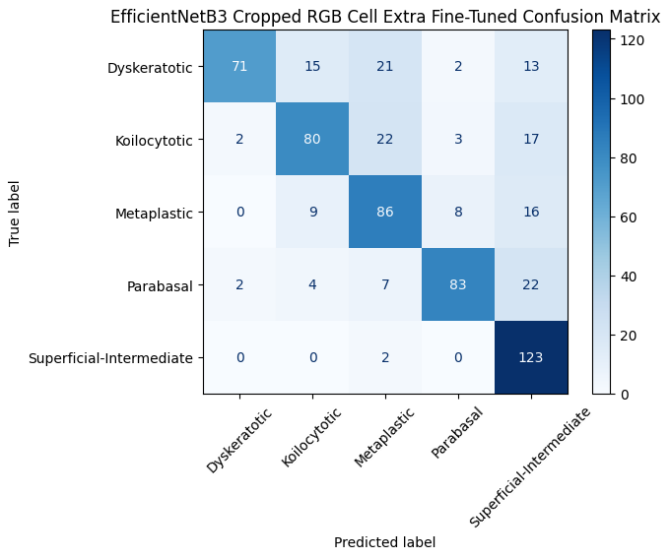


Fig. 45: EfficientNetB3 Confusion Matrix

The EfficientNetB3 model showed consistent reductions in both the training and the validation loss throughout the feature extraction and fine-tuning phases, showing effective learning from the segmented cell images. It hit its lowest validation loss post-fine-tuning. Yet, in the extra fine-tuning phase, the validation loss spiked significantly while training loss kept inching down. This implies the model started to overfit; more training at this point didn't help it generalize better.

Looking at the confusion matrix, EfficientNetB3 really shined on the Superficial-Intermediate class, nailing most classifications there. It also did well on Koilocytotic, Metaplastic, and Parabasal classes. Still, some mix-ups occurred between Dyskeratotic, Metaplastic, and Superficial-Intermediate cells. Overall, the model delivered strong classification performance, though that uptick in validation loss shows extra training after initial fine-tuning wasn't helpful.

TABLE XXIII: Comparison of CNN Models on Cropped RGB Cell Dataset

Model Stage	Train Loss	Val Loss	Test Acc.	Macro F1	Epoch
ResNet50 Fine-Tuned	0.0519	0.1208	0.9276	0.9278	10
EfficientNetB3 Fine-Tuned	0.1900	0.2754	0.9095	0.9093	10
ResNet50 Feature Extraction	0.2488	0.2440	0.8964	0.8955	5
EfficientNetB3 Feature Extraction	0.3852	0.4175	0.8553	0.8518	5
DenseNet121 Fine-Tuned	0.5467	0.5342	0.7730	0.7732	10
MobileNetV2 Fine-Tuned	0.5025	0.5724	0.7697	0.7730	10
EfficientNetB3 Extra Fine-Tuned	1.3153	1.4384	0.7286	0.7268	13
MobileNetV2 Feature Extraction	0.6415	0.6676	0.7155	0.7181	5
DenseNet121 Feature Extraction	0.7889	0.7864	0.6891	0.6786	5

This table looks at how different CNN models performed when trained on the cropped RGB cell dataset. Fine-tuning generally boosted all the models' performance over just extracting features. ResNet50 came out on top, hitting a test accuracy of 92.76% and a Macro F1-score of 0.9278 after being fine-tuned. EfficientNetB3 did pretty well too, with a test accuracy of 90.95%, while DenseNet121 and MobileNetV2 had lower scores around 77%. Deeper networks like ResNet50 and EfficientNetB3 must have picked up on more distinctive traits from the segmented cell pictures.

Also, the table reveals that extra training didn't always make things better. Even though they added an extra fine-tuning step for EfficientNetB3, unfreezing more layers caused its accuracy to drop to 72.86%. Looks like the model might've begun to overfit the training data. The same goes for tracking the training and validation loss across the models. In the end, the CNN models grasped the cropped RGB cell representation effectively, and ResNet50 struck the right balance between training performance and how it did on the test set.

3) Comparison of Feature-Based and Image-Based Deep Learning Models:

D. Grad-CAM

TABLE XXIV: Model-level Grad-CAM attention summary for non-segmented test images

Model	Test Images	Accuracy (%)	Cell Att. (%)	Bg. Att. (%)	Hot Inside Cell (%)
ResNet50	608	92.76	86.55	13.29	90.74
EfficientNetB3	608	90.95	85.60	14.40	91.29
DenseNet121	608	77.30	70.83	24.40	71.60
MobileNetV2	608	76.97	80.49	19.51	87.18
EfficientNetB3_Extra	608	72.86	86.21	13.79	93.07

TABLE XXV: Grad-CAM comparison of correct and wrong predictions on cropped RGB cell images

Model	Prediction Type	Num. Images	Mean Conf.	Cell Att. (%)	Bg. Att. (%)	Hot Inside Cell (%)
DenseNet121	Wrong	138	0.7152	0.7265	0.2155	0.7622
DenseNet121	Correct	470	0.8883	0.7030	0.2523	0.7024
EfficientNetB3	Wrong	55	0.6547	0.7726	0.2274	0.8414
EfficientNetB3	Correct	553	0.9215	0.8643	0.1357	0.9200
EfficientNetB3_Extra	Wrong	165	0.4017	0.8524	0.1476	0.9026
EfficientNetB3_Extra	Correct	443	0.5918	0.8657	0.1343	0.9412
MobileNetV2	Wrong	140	0.6562	0.8279	0.1721	0.9060
MobileNetV2	Correct	468	0.8399	0.7980	0.2020	0.8616
ResNet50	Wrong	44	0.7241	0.7946	0.2054	0.8167
ResNet50	Correct	564	0.9620	0.8710	0.1272	0.9145

Correct Grad-CAM Examples on Cropped Segmented RGB Cell Images

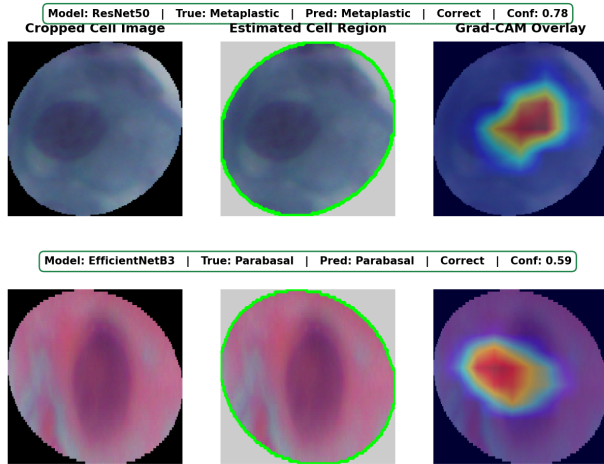


Fig. 46: Correct Guess Visualization with GradCAM

Wrong Grad-CAM Examples on Cropped Segmented RGB Cell Images

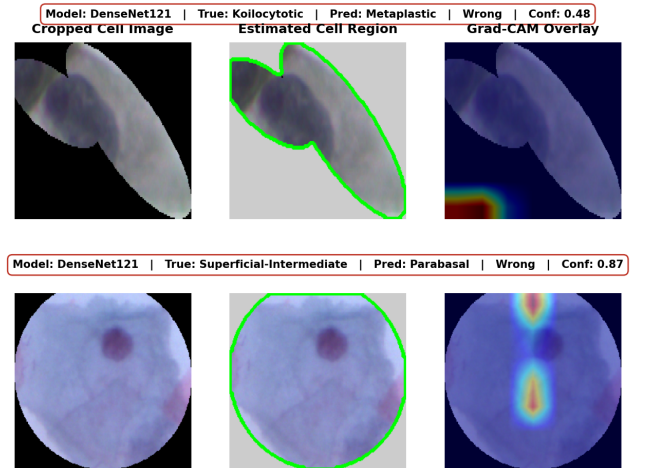


Fig. 47: Wrong Guess Visualization with GradCAM

Table XXIV presents the Grad-CAM attention statistics for all CNN models. The results show that the models generally focused on the cell region rather than the background during classification. ResNet50 achieved the highest classification accuracy of 92.76%, while also maintaining a high cell attention value of 86.55% and a low background attention value of 13.29%. EfficientNetB3 produced similar attention behavior and reached 90.95% accuracy. DenseNet121 and MobileNetV2 achieved lower classification performance, although their attention maps still remained mostly concentrated inside the cell region. Interestingly, EfficientNetB3-Extra had the highest Hot Inside Cell value of 93.07%, indicating that its strongest activations were concentrated within the cell area, but its overall classification accuracy was lower. This suggests that focusing strongly on the cell region alone is not sufficient for achieving the best classification performance.

Table XXV compares Grad-CAM statistics for correct and incorrect predictions. For most models, correct predictions have higher confidence values and slightly higher attention concentrated inside the cell region. However, the differences between correct and incorrect predictions are not always very large. For example, EfficientNetB3-Extra shows similar attention distributions for both correct and wrong predictions despite having a lower overall accuracy. This indicates that some errors are caused by visual similarity between classes rather than the model attending to irrelevant image regions. In other words, the model often looks at the correct area of the image but still struggles to distinguish between certain cell types.

The Grad-CAM visualizations support these observations. In the correctly classified examples, the activation maps are concentrated on important cell structures and remain mostly

inside the estimated cell boundary. This suggests that the models learn meaningful morphological features from the segmented cell images. In the incorrectly classified examples, two different behaviors can be observed. In some cases, the activation shifts toward less informative regions of the image, while in other cases the model still focuses on the cell but predicts the wrong class. This is likely due to similarities in shape, texture, or staining characteristics between different cell categories. Overall, the Grad-CAM analysis indicates that the CNN models base their decisions primarily on the cell itself rather than the background, which increases the reliability and interpretability of the classification results.

IV. LIMITATIONS

This study uses a combo of k-means clustering and ellipse fitting for segmentation. While the average Dice scores – nucleus: 0.81 and cytoplasm: 0.85 – are decent, some images fail to segment correctly, especially Superficial-Intermediate cells. The nucleus detection there tends to fall short sometimes.

The dataset includes 4049 images, which is quite small for deep learning. Because of that, the team used transfer learning with ImageNet pretrained weights. However, this choice might prevent the model from picking up on certain medical-specific details.

Another issue is that the Koilocytotic and Metaplastic cells look very similar. They share colors, textures, and shapes, confusing most models, including those based on segmentation. Sorting out these two groups continues to be tough.

Lastly, the Grad-CAM analysis relied on auto-generated cell regions instead of accurate manual masks. As such, the insights drawn from this process should be seen as rough estimates.

V. CONCLUSION

In this study, classical machine learning and deep learning approaches were comparatively evaluated on the SIPaKMeD dataset under both segmented and non-segmented settings. Among non-segmented classical ML models, SVM achieved 94.90% accuracy using all 219 features. After applying segmentation, Random Forest reached 97.37%, demonstrating that segmentation significantly improves feature quality. In the deep learning experiments, EfficientNetB3 with extra fine-tuning achieved the best result of 94.57% on non-segmented images, while the segmented feature-based Basic MLP reached 96.71% accuracy. Among CNN models trained on segmented images, ResNet50 performed best with 92.76% accuracy. Grad-CAM analyses confirmed that the models largely focused on biologically meaningful cell regions, and revealed that some incorrect predictions were caused by attention shifting toward the background, while others stemmed from confusion between visually similar classes. Segmentation-based feature extraction boosts both classical ML and deep learning models' performance. Still, CNNs trained on segmented images did worse than those using full images. This means context from whole pictures matters a lot too. Looking ahead, using fancier

segmentation techniques and bigger datasets should solidify these results even more.

REFERENCES

- [1] M. E. Plissiti, C. Dimitrakopoulos, G. Sfikas, C. Nikou, O. Krikoni, and A. Charchanti, "SIPaKMeD: A new dataset for feature and image based classification of normal and pathological cervical cells in Pap smear images," *IEEE International Conference on Image Processing (ICIP)*, 2018. [Online]. Available: <https://www.cs.uoi.gr/~marina/sipakmed.html>
- [2] A. Rosebrock, "Why is my validation loss lower than my training loss?," *PyImageSearch*, Oct. 14, 2019. [Online]. Available: <https://pyimagesearch.com/2019/10/14/why-is-my-validation-loss-lower-than-my-training-loss/>